ЧАСТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«СУРГУТСКИЙ КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА» (ЧПОУ «СКП»)

КУРСОВОЙ ПРОЕКТ

Тема: «Проектирование и разработка модуля учета движения канцелярских товаров»

ПО ПМ.01 РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ

МДК.01.01 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

Специальность 09.02.07 «Информационные системы и программирование»

Выполнил:
студент группы ИСиП-23/1
[Фамилия Имя Отчество]

Руководитель:
Трушина Александра Игоревна

Сургут, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ....3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ....5
1.1 Учёт движения канцелярских товаров в организации....5
1.2 Обзор аналогов....6
1.3 Требования к модулю....6
1.4 Выбор средств разработки....8
Выводы по разделу 1....9
2 ПРОЕКТИРОВАНИЕ МОДУЛЯ....10
2.1 Модель данных....10
2.2 Главное правило и расчёт остатка....12
2.3 Архитектура модуля....13
2.4 Варианты использования....14
2.5 Алгоритм оформления выдачи....15
Выводы по разделу 2....16
3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ МОДУЛЯ....17
3.1 Справочник товаров....17
3.2 Поступление и выдача....19
3.3 Расчёт остатка и контроль минимума....21
3.4 История движения и итоги за период....22
3.5 Хранение данных и развёртывание....24
3.6 Тестирование программного модуля....25
Выводы по разделу 3....31
ЗАКЛЮЧЕНИЕ....32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....34
ПРИЛОЖЕНИЕ А. ЛИСТИНГ АВТОМАТИЧЕСКИХ ТЕСТОВ....36
ПРИЛОЖЕНИЕ Б. ПЕРЕЧЕНЬ АВТОМАТИЧЕСКИХ ТЕСТОВ....49
ПРИЛОЖЕНИЕ В. ТАБЛИЦА РУЧНЫХ ПРОВЕРОК....52
ПРИЛОЖЕНИЕ Г. ЛИСТИНГ КЛЮЧЕВЫХ КЛАССОВ....55
ПРИЛОЖЕНИЕ Д. ИНСТРУКЦИЯ ЗАПУСКА....60
ПРИЛОЖЕНИЕ Е. ИНСТРУКЦИЯ ПОЛЬЗОВАТЕЛЯ....61

ВВЕДЕНИЕ

В любой организации ежедневно расходуются канцелярские товары: бумага, ручки, папки, скрепки, картриджи. Их закупают партиями у поставщиков и выдают сотрудникам по мере необходимости. Если учёт ведётся в тетради или в разрозненных таблицах, кладовщик не может быстро ответить на простые вопросы: сколько бумаги осталось на складе, кто и когда получил степлеры, какие товары пора заказывать. Ошибки в ручном пересчёте приводят к тому, что товар заканчивается неожиданно, а сотрудники вынуждены ждать новую поставку.

Актуальность темы определяется тем, что учёт материальных запасов – обязательная часть хозяйственной деятельности организации, а для небольших подразделений полноценные учётные системы избыточны и дороги. Требуется простой модуль, который хранит историю движения товаров, сам считает остаток и предупреждает о необходимости пополнения.

Степень разработанности проблемы. Задачи складского учёта решаются как крупными учётными системами (например, «1С:Бухгалтерия», облачный сервис «МойСклад»), так и электронными таблицами. Первые требуют лицензий, настройки и обучения, вторые не защищают от ошибок ввода: в таблице можно случайно выдать больше товара, чем есть, или изменить уже внесённую запись. Поэтому разработка небольшого специализированного модуля с жёсткими правилами проверки данных остаётся практически значимой.

Объект исследования – процесс учёта поступления и выдачи канцелярских товаров в организации.

Предмет исследования – программный модуль учёта движения канцелярских товаров с автоматическим расчётом остатка и контролем минимального количества.

Цель работы – спроектировать и разработать модуль учёта движения канцелярских товаров на платформе .NET 10, который ведёт справочник товаров, регистрирует операции поступления и выдачи, автоматически рассчитывает

текущий остаток, хранит историю движения и контролирует минимальное количество товара.

Для достижения цели поставлены задачи:

– проанализировать предметную область и сформулировать требования к модулю;

– выбрать и обосновать средства разработки;

– спроектировать модель данных, архитектуру и алгоритм главной операции;

– реализовать справочник товаров, операции поступления и выдачи, расчёт остатка, историю движения и контроль минимума;

– разработать автоматические тесты и выполнить ручные проверки всех обязательных функций;

– подготовить инструкцию запуска и инструкцию пользователя.

Методы исследования: анализ предметной области и аналогов, объектно-ориентированное проектирование, моделирование данных (ER-диаграмма), модульное тестирование, ручное функциональное тестирование.

Теоретическая значимость работы состоит в систематизации правил учёта движения товаров в виде формализованных ограничений и формулы расчёта остатка. Практическая значимость – модуль готов к использованию в небольшой организации или учебном заведении: он запускается одной командой, хранит данные в файле базы SQLite и не требует отдельного сервера базы данных.

Работа состоит из введения, трёх разделов, заключения, списка использованных источников и приложений. В первом разделе проанализирована предметная область и сформулированы требования, во втором описано проектирование модуля, в третьем – реализация, развёртывание и тестирование.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Учёт движения канцелярских товаров в организации

Канцелярские товары относятся к материально-производственным запасам организации. Порядок их учёта определяется федеральным законом «О бухгалтерском учёте» [1] и федеральным стандартом ФСБУ 5/2019 «Запасы» [2]: каждое поступление и каждое списание должно подтверждаться документом, а остаток на любую дату должен быть восстановим по первичным записям.

На практике движение канцтоваров выглядит так. Офис-менеджер или кладовщик получает товар от поставщика по товарной накладной или универсальному передаточному документу (далее – УПД) и фиксирует поступление. Затем по заявкам выдаёт товар сотрудникам разных отделов и фиксирует, кто, что и в каком количестве получил. Периодически ответственный проверяет, каких позиций осталось мало, и формирует заказ поставщику.

Основные участники процесса и их задачи приведены в таблице 1.

Таблица 1 – Участники процесса учёта канцелярских товаров

Участник	**Действия**	**Что ему нужно от модуля**
Кладовщик (офис-менеджер)	принимает товар, выдаёт сотрудникам, заказывает пополнение	быстрое оформление операций, актуальный остаток, список товаров к заказу
Сотрудник отдела	получает канцтовары	(косвенно) чтобы нужный товар был в наличии
Руководитель подразделения	контролирует расход	итоги поступлений и выдач за период

Без автоматизации в этом процессе возникают типичные проблемы:

– остаток пересчитывается вручную и легко расходится с фактическим количеством;

– можно записать выдачу большего количества, чем есть на складе, и обнаружить ошибку только при инвентаризации;

– нет единой истории: чтобы узнать, кто получил товар месяц назад, приходится перелистывать журнал;

– о том, что товар заканчивается, узнают, когда он уже закончился;

– исправление ошибки «зачёркиванием» уничтожает историю, и остаток становится невозможно проверить.

Из этого следует главное требование к модулю: остаток не должен вводиться руками, он должен вычисляться по истории операций, а операции, нарушающие правила, должны отклоняться до сохранения.

1.2 Обзор аналогов

Рассмотрены три распространённых способа учёта: учётная система «1С:Бухгалтерия», облачный сервис «МойСклад» и электронная таблица Microsoft Excel. Результаты сравнения приведены в таблице 2.

Таблица 2 – Сравнение аналогов

Критерий	**«1С:Бухгалтерия»**	**«МойСклад»**	**Таблица Excel**	**Разрабатываемый модуль**
Автоматический расчёт остатка	да	да	только формулами, вручную	да
Запрет выдачи сверх остатка	настраивается	да	нет	да
Контроль минимального остатка	да	да	нет	да
История движения	да	да	частично	да
Стоимость	платная лицензия	подписка	входит в офисный пакет	бесплатно
Сложность освоения	высокая	средняя	низкая	низкая
Работа без интернета и сервера	да (локальная установка)	нет	да	да

Полнофункциональные системы решают задачу, но избыточны для учёта одной группы товаров в небольшом подразделении. Электронная таблица проста, но не защищает данные от ошибок. Разрабатываемый модуль занимает промежуточное положение: он содержит только нужные функции, но строго проверяет каждую операцию.

1.3 Требования к модулю

Функциональные требования сформулированы на основе задания: «Реализовать справочник товаров, операции поступления и выдачи,

автоматический расчёт текущего остатка и историю движения. Добавить контроль минимального количества товара». Перечень функций приведён в таблице 3.

Таблица 3 – Функциональные требования

Код	Функция	Содержание
Ф1	Справочник товаров	добавление, просмотр, изменение, архивирование; поиск по наименованию и артикулу
Ф2	Поступление	количество, дата, поставщик или номер документа, комментарий
Ф3	Выдача	количество, дата, сотрудник-получатель, комментарий; запрет выдачи сверх остатка
Ф4	Расчёт остатка	остаток вычисляется по операциям и не вводится вручную
Ф5	История движения	список операций с фильтрами по товару, типу, сотруднику, периоду; история в карточке товара
Ф6	Контроль минимума	пометка «Требует пополнения», фильтр «Ниже минимума», предупреждение при выдаче
Ф7	Итоги за период	сколько поступило и сколько выдано по каждому товару
Ф8	Справочник сотрудников	добавление и изменение получателей

Ограничения предметной области:

– количество в операции – целое число больше нуля;

– для выдачи обязателен сотрудник-получатель, для поступления – поставщик или номер документа;

– дата операции не может быть в будущем;

– с архивным товаром операции не проводятся;

– артикул товара уникален, минимальный остаток не меньше нуля;

– операции не редактируются и не удаляются: ошибка исправляется новой корректирующей операцией с комментарием.

Нефункциональные требования:

– платформа .NET 10, язык C#, веб-приложение ASP.NET Core MVC;

– хранение данных в файле базы SQLite, база создаётся автоматически при запуске;

– все надписи и сообщения об ошибках на русском языке;

– неверные данные и обращение к несуществующей записи не приводят к аварийному завершению;

– предметная логика отделена от контроллеров и проверяется автоматическими тестами;

– исходный код хранится в системе контроля версий Git.

1.4 Выбор средств разработки

Средства разработки выбраны с учётом изученного на занятиях материала и требований к модулю (таблица 4).

Таблица 4 – Средства разработки

Средство	Версия	Назначение и причина выбора
C#	14	основной язык: строгая типизация, LINQ для запросов к данным [5]
.NET SDK	10.0	платформа с долгосрочной поддержкой (LTS), кроссплатформенная
ASP.NET Core MVC	10.0	веб-приложение по шаблону «модель – представление – контроллер» [4], [10]
Entity Framework Core	10.0.15	объектно-реляционное отображение, миграции базы [6], [11]
SQLite	3 (через Microsoft.EntityFrameworkCore.Sqlite)	встраиваемая база в одном файле, не требует сервера [13]
Bootstrap	5.3.3	оформление страниц без собственного CSS-фреймворка
xUnit	2.9.3	автоматические тесты [12]
Git, GitHub	2.x	контроль версий, история этапов, автоматическая сборка и тесты (GitHub Actions) [8]

Выбор ASP.NET Core MVC вместо Web API обусловлен тем, что пользователю модуля – кладовщику – нужны готовые страницы с формами и таблицами, а не программный интерфейс. При этом для ручных проверок HTTP-запросами в модуль добавлены несколько API-методов, возвращающих данные в формате JSON.

SQLite выбрана потому, что модуль рассчитан на одного пользователя или небольшой отдел: база хранится в одном файле, её легко скопировать и восстановить, а EF Core позволяет при необходимости перейти на другую СУБД без изменения предметной логики.

Выводы по разделу 1

В разделе проанализирован процесс учёта канцелярских товаров и выявлены проблемы ручного учёта: расхождение остатка, выдача сверх наличия, отсутствие истории и несвоевременное пополнение. Сравнение аналогов показало, что для небольшого подразделения нужен простой модуль со строгими проверками. Сформулированы восемь функциональных требований, ограничения предметной области и нефункциональные требования, выбраны средства разработки: C#, .NET 10, ASP.NET Core MVC, EF Core, SQLite, xUnit и Git.

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ

2.1 Модель данных

Модель данных состоит из трёх сущностей: товар (Product), сотрудник (Employee) и операция движения (StockOperation). Товар и сотрудник – справочники, операция – журнал, в который добавляются записи о поступлениях и выдачах. Связи и поля показаны на рисунке 1 в соответствии с ER-диаграммой.

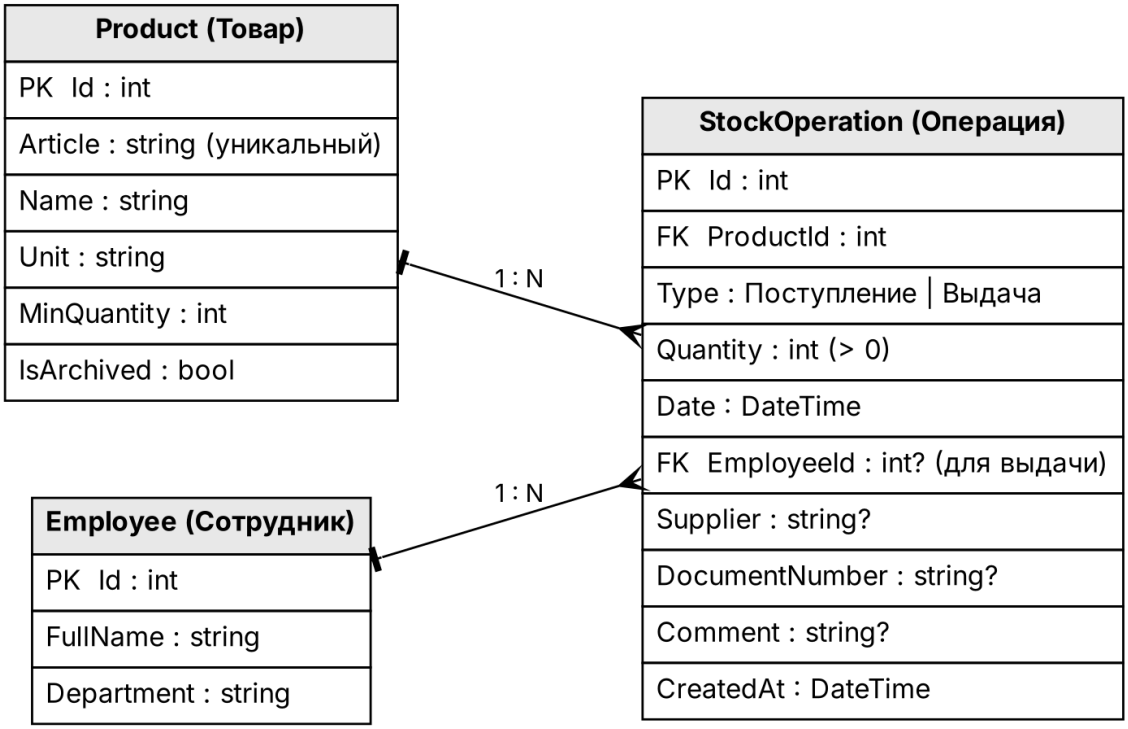


Рисунок 1 – ER-диаграмма модели данных

Описание сущностей приведено в таблице 5.

Таблица 5 – Сущности модели данных

Сущность	Поля	Ограничения
Product (товар)	Id, Article, Name, Unit, MinQuantity, IsArchived	артикул обязателен и уникален (уникальный индекс); MinQuantity ≥ 0; товар не удаляется, а архивируется
Employee (сотрудник)	Id, FullName, Department	ФИО и отдел обязательны; сотрудник не удаляется, так как на него ссылаются выдачи
StockOperation (операция)	Id, ProductId, Type, Quantity, Date, EmployeeId, Supplier, DocumentNumber, Comment, CreatedAt	Quantity > 0; EmployeeId обязателен для выдачи; Supplier или DocumentNumber обязателен для поступления; запись не изменяется

Связь «товар – операции» имеет тип «один ко многим»: у товара может быть сколько угодно операций, каждая операция относится ровно к одному товару. Связь «сотрудник – операции» также «один ко многим», но необязательная: у поступлений сотрудника нет. Для обеих связей задано правило удаления Restrict – база не позволит удалить товар или сотрудника, у которых есть операции. Это защищает историю движения.

Сущности описаны классами C#, а ограничения ввода – атрибутами проверки данных. Класс товара приведён в листинге 1: атрибут [Required] делает поле обязательным, [StringLength] ограничивает длину, [Range] задаёт допустимый диапазон, а [Display] – русскую подпись поля на странице.

Листинг 1 – Класс сущности «Товар» (Models/Product.cs, сокращённо)

```
public class Product
{
    public int Id { get; set; }

    [Display(Name = "Артикул")]
    [Required(ErrorMessage = "Укажите артикул")]
    [StringLength(20, ErrorMessage = "Артикул — не длиннее 20 символов")]
    public string Article { get; set; } = string.Empty;

    [Display(Name = "Наименование")]
    [Required(ErrorMessage = "Укажите наименование")]
    public string Name { get; set; } = string.Empty;

    [Display(Name = "Минимальный остаток")]
    [Range(0, 100000, ErrorMessage = "Минимальный остаток должен быть от 0 до
100000")]
    public int MinQuantity { get; set; }

    public bool IsArchived { get; set; }          // архив вместо удаления
    public List<StockOperation> Operations { get; set; } = new();   // связь
1 — N
}
```

Связи и уникальность артикула настраиваются в контексте базы данных (листинг 2). Метод OnModelCreating вызывается EF Core при построении модели, и по нему же создаётся миграция.

Листинг 2 – Настройка связей и индексов (Data/AppDbContext.cs)

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Артикул уникален — база сама не даст записать два одинаковых
    modelBuilder.Entity<Product>().HasIndex(p => p.Article).IsUnique();

    // Товар 1 — N операции; Restrict: нельзя удалить товар, у которого есть
операции
    modelBuilder.Entity<StockOperation>()
        .HasOne(o => o.Product).WithMany(p => p.Operations)
        .HasForeignKey(o => o.ProductId).OnDelete(DeleteBehavior.Restrict);

    // Сотрудник 1 — N операции (выдачи)
    modelBuilder.Entity<StockOperation>()
        .HasOne(o => o.Employee).WithMany(e => e.Operations)
        .HasForeignKey(o => o.EmployeeId).OnDelete(DeleteBehavior.Restrict);
}
```

Тип операции хранится перечислением OperationType со значениями Receipt (поступление) и Issue (выдача). В таблице операций нет поля «остаток» – это принципиальное проектное решение, следующее из главного правила.

2.2 Главное правило и расчёт остатка

Главное правило модуля: нельзя выдать больше товара, чем есть на складе. Остаток при этом не хранится, а вычисляется по формуле (1):

$$O = \Sigma Q_{пост} - \Sigma Q_{выд} \quad (1)$$

где O – текущий остаток товара;

$\Sigma Q_{пост}$ – сумма количеств во всех операциях поступления данного товара;

$\Sigma Q_{выд}$ – сумма количеств во всех операциях выдачи данного товара.

Выдача количества q разрешается, если выполняется условие (2):

$$q \le O \quad (2)$$

где q – запрошенное количество для выдачи.

Признак «Требует пополнения» устанавливается по условию (3):

$$O \le O_{min} \quad (3)$$

где Omin – минимальный остаток товара (поле MinQuantity).

Из формул следуют граничные случаи, которые проверяются тестами: выдача ровно всего остатка допустима (остаток становится 0); выдача «остаток + 1» отклоняется; остаток, равный минимуму, уже означает «Требует

пополнения»; у нового товара без операций остаток равен нулю. Если после выдачи остаток опускается до минимума или ниже, выдача выполняется, но пользователь получает предупреждение.

2.3 Архитектура модуля

Модуль построен по многослойной архитектуре (рисунок 2). Каждый слой выполняет свою задачу и обращается только к нижележащему.

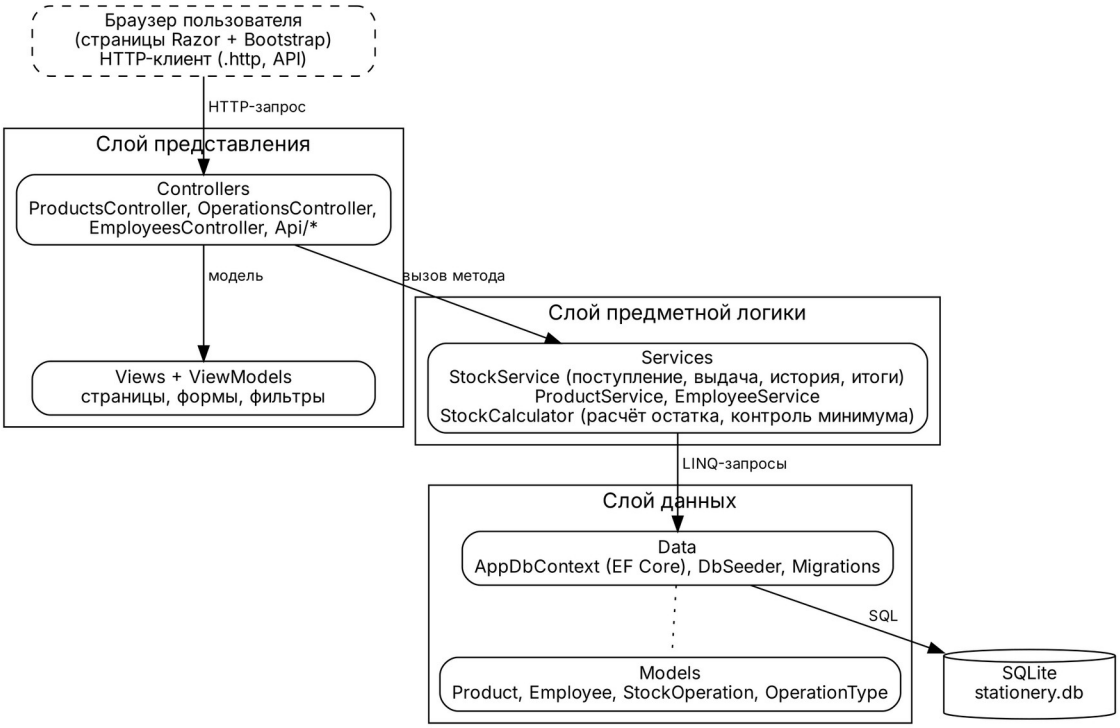


Рисунок 2 – Архитектура модуля

Слой представления – контроллеры и представления Razor. Контроллер принимает HTTP-запрос, вызывает метод сервиса и выбирает страницу для ответа; в нём нет ни расчётов, ни обращений к базе. Слой предметной логики – классы папки Services: StockService (поступление, выдача, история, итоги), ProductService, EmployeeService и статический класс StockCalculator с чистыми функциями расчёта. Слой данных – контекст AppDbContext, миграции и начальные учебные данные (DbSeeder).

Сервисы возвращают результат в виде объекта OperationResult со свойствами Success, Error, Warning и IsNotFound. Ошибка пользователя – это не исключение, а обычный результат с понятным текстом на русском языке,

который контроллер показывает на странице. Назначение папок проекта приведено в таблице 6.

Таблица 6 – Структура проекта

Папка	Содержимое
Models	сущности Product, Employee, StockOperation, перечисление OperationType, список единиц измерения Units
Data	AppDbContext, DbSeeder (учебные данные), Migrations (миграция InitialCreate)
Services	StockService, ProductService, EmployeeService, StockCalculator, OperationResult
ViewModels	формы и фильтры: ReceiptForm, IssueForm, HistoryFilter, ProductFilter, модели страниц
Controllers	ProductsController, OperationsController, EmployeesController, HomeController; Api – JSON-методы
Views	страницы Razor и общие части (_Layout, _OperationsTable, формы)
StationeryAccounting.Tests	автоматические тесты xUnit

2.4 Варианты использования

Пользователь модуля – кладовщик (офис-менеджер). Его действия показаны на диаграмме вариантов использования в соответствии с рисунком 3. Вариант «Оформить выдачу» всегда включает проверку остатка.

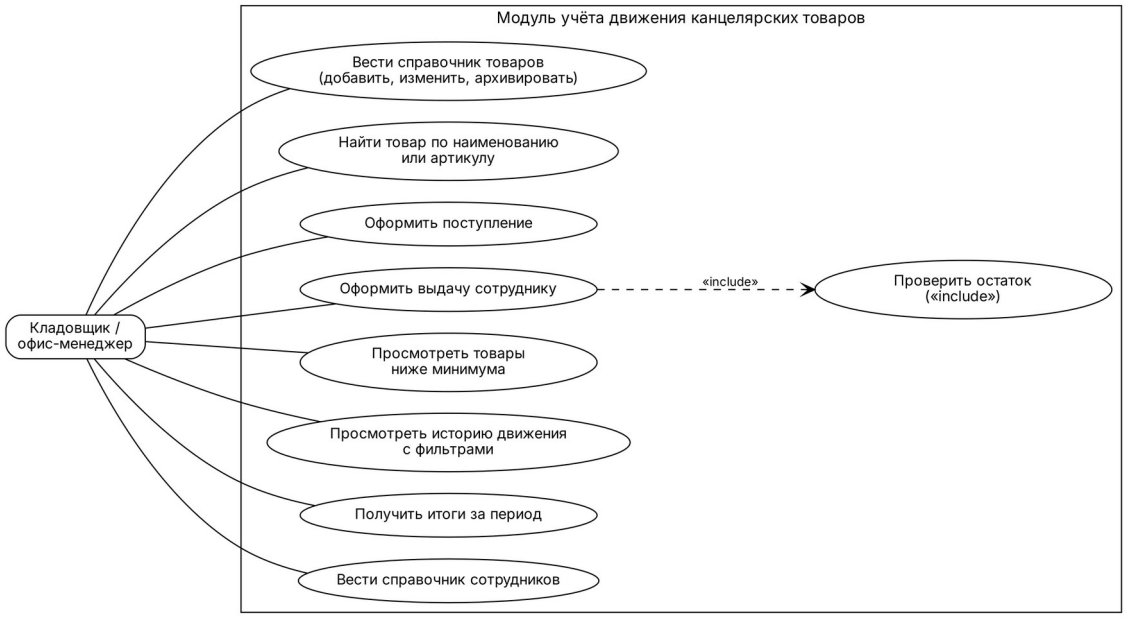


Рисунок 3 – Диаграмма вариантов использования

Роли и вход в систему заданием не предусмотрены: модуль рассчитан на одного ответственного сотрудника, поэтому авторизация не реализовывалась. Это отмечено в ограничениях модуля.

2.5 Алгоритм оформления выдачи

Выдача – главная операция модуля. Порядок проверок показан на блок-схеме в соответствии с рисунком 4 (оформлена по ГОСТ 19.701-90 [3]). Сначала проверяются данные, которые не требуют обращения к базе (количество и дата), затем наличие получателя, существование и состояние товара, и только после этого рассчитывается остаток. Такой порядок позволяет как можно раньше отклонить неверный запрос.

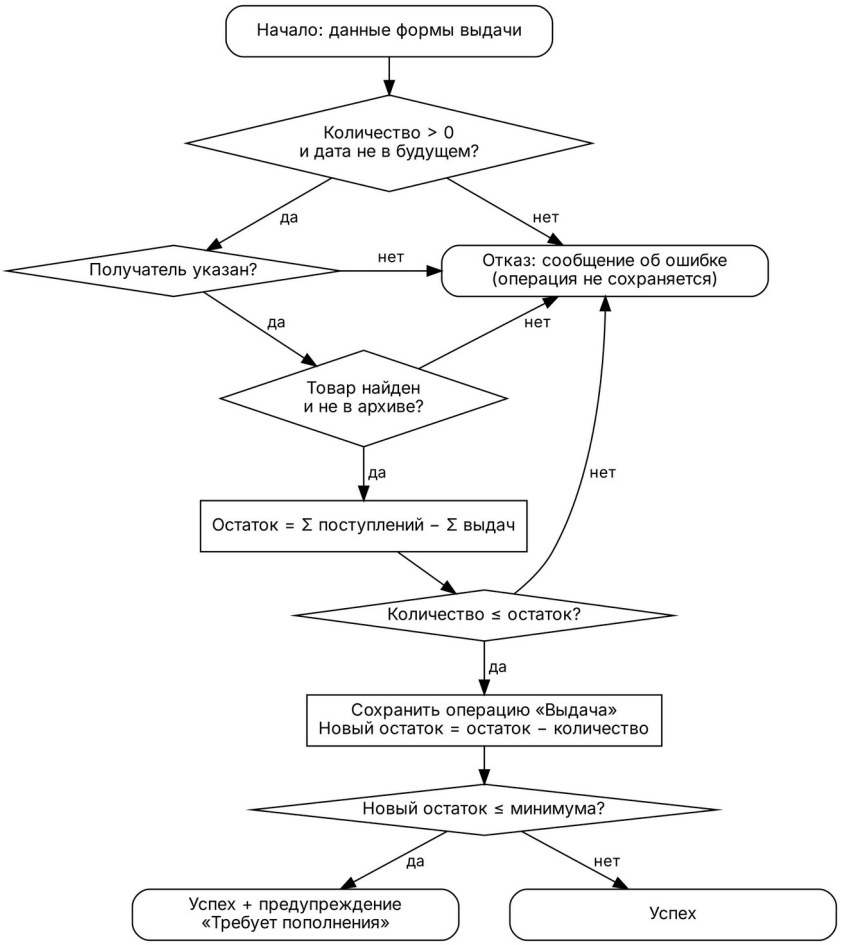


Рисунок 4 – Блок-схема алгоритма оформления выдачи

Проверка остатка и сохранение выдачи выполняются внутри критической секции (оператор lock). Если два пользователя одновременно оформят выдачу одного товара, вторая операция начнёт проверку только после сохранения

первой и увидит уже уменьшенный остаток. Без этого обе операции могли бы пройти проверку и вместе выдать больше, чем есть на складе.

Выводы по разделу 2

Спроектирована модель данных из трёх сущностей со связями «один ко многим»; остаток не хранится, а вычисляется по формуле (1), что исключает его расхождение с историей. Сформулированы условия выдачи и пополнения в виде формул (2) и (3) и выделены граничные случаи. Выбрана трёхслойная архитектура с отделением предметной логики от контроллеров, построены диаграмма вариантов использования и блок-схема главной операции с защитой от одновременных выдач.

3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ МОДУЛЯ

3.1 Справочник товаров

Справочник товаров реализован контроллером ProductsController и сервисом ProductService. На главной странице справочника, представленной на рисунке 5, для каждого товара выводятся артикул, наименование, единица измерения, текущий остаток и минимальный остаток. Строки товаров, требующих пополнения, выделены красным цветом и помечены надписью «Требует пополнения».

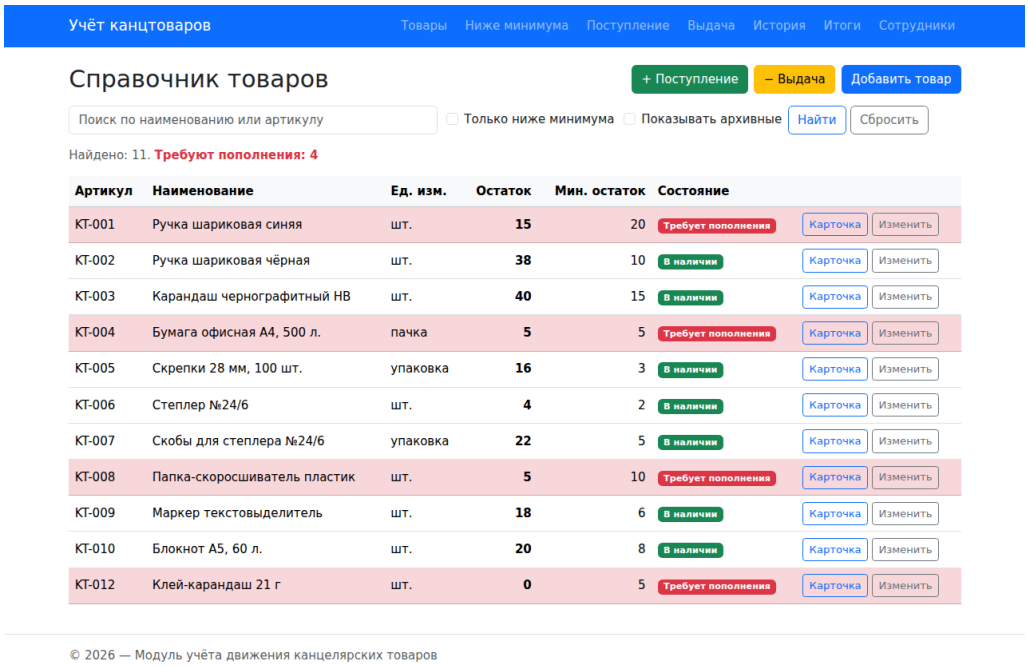


Рисунок 5 – Справочник товаров с текущим остатком

Остатки для списка рассчитываются одним запросом: операции группируются по товару, и для каждой группы вызывается функция StockCalculator.CalculateBalance. Поиск выполняется по части наименования или артикула без учёта регистра (листинг 3):

Листинг 3 – Поиск в справочнике товаров (Services/ProductService.cs)

```
if (!string.IsNullOrWhiteSpace(filter.Search))
{
    string text = filter.Search.Trim();
    items = items.Where(i =>
        i.Name.Contains(text, StringComparison.OrdinalIgnoreCase) ||
        i.Article.Contains(text, StringComparison.OrdinalIgnoreCase));
}
```

Поиск выполняется в памяти, а не средствами базы: в SQLite сравнение без учёта регистра работает только для латинских букв, а наименования товаров записаны по-русски. Для справочника из десятков и сотен позиций это не влияет на скорость.

Пример. Пользователь вводит в строку поиска «ручка». Метод Contains с параметром OrdinalIgnoreCase сравнивает строки без учёта регистра, поэтому находятся «Ручка шариковая синяя» и «Ручка шариковая чёрная», а запрос «kt-003» находит «Карандаш чернографитный НВ» по артикулу КТ-003.

При добавлении и изменении товара сервис убирает лишние пробелы, приводит артикул к верхнему регистру и проверяет его уникальность среди других товаров. Попытка добавить товар с артикулом «kt-001», который уже существует как «КТ-001», отклоняется с сообщением, изображённым на рисунке 6.

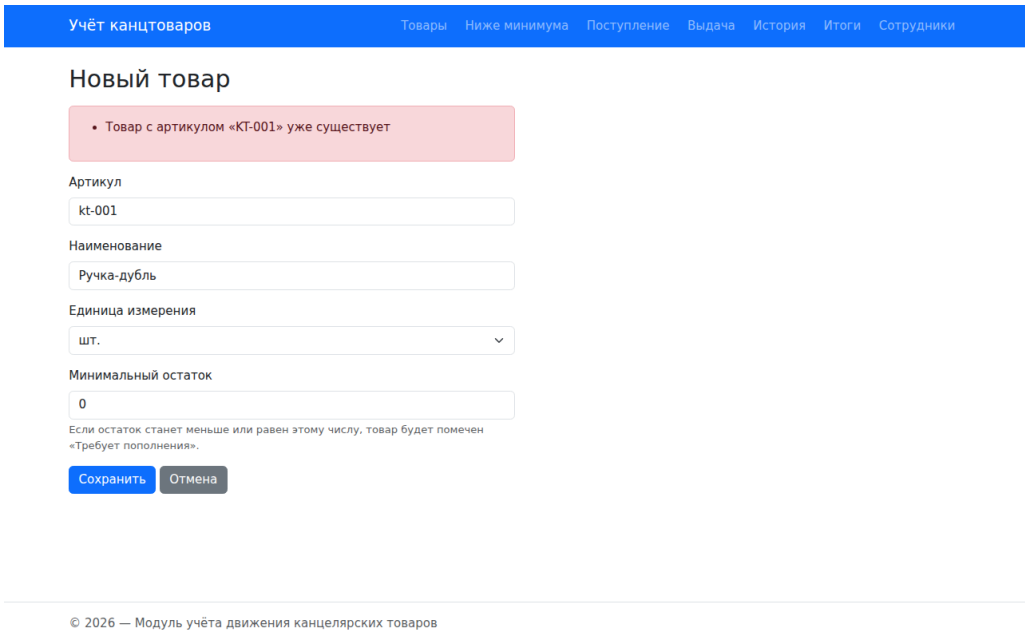


Рисунок 6 – Отклонение повторяющегося артикула

Товар не удаляется: вместо удаления используется архивирование. Архивный товар скрыт в справочнике по умолчанию, не предлагается в формах операций, а попытка провести операцию с ним через API отклоняется. Так сохраняется история движения, по которой рассчитывается остаток.

3.2 Поступление и выдача

Операции реализованы в классе StockService. Фрагмент метода выдачи, содержащий главное правило, приведён в листинге 4.

Листинг 4 – Проверка остатка при выдаче (Services/StockService.cs, метод Issue)

```
lock (IssueLock)
{
    // ГЛАВНОЕ ПРАВИЛО: нельзя выдать больше, чем есть на складе
    int balance = GetBalance(product.Id);
    if (form.Quantity > balance)
    {
        string unit = Units.Short(product.Unit);
        return OperationResult.Fail(
            $"Недостаточно товара «{product.Name}»: на складе {balance}
{unit}, " +
            $"запрошено {form.Quantity} {unit}");
    }
    _db.StockOperations.Add(new StockOperation { ... });
    _db.SaveChanges();

    int newBalance = balance - form.Quantity;
    string? warning = null;
    if (StockCalculator.NeedsRestock(newBalance, product.MinQuantity))
        warning = $"Внимание: остаток «{product.Name}» — {newBalance} ...
Товар требует пополнения.";
    return OperationResult.Ok(warning);
}
```

Пример. На складе 38 шт. товара КТ-002 «Ручка шариковая чёрная». Пользователь оформляет выдачу 100 шт. Метод GetBalance возвращает 38, условие 100 > 38 истинно, поэтому метод возвращает отказ, операция не сохраняется, а на форме выдачи показывается сообщение «Недостаточно товара „Ручка шариковая чёрная“: на складе 38 шт., запрошено 100 шт.» (рисунок 7).

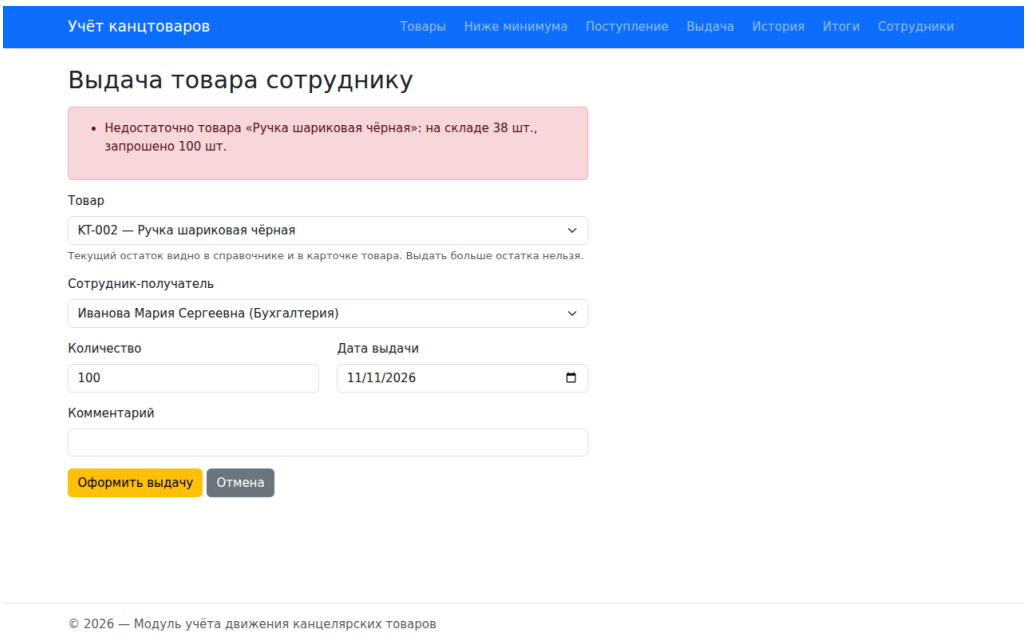


Рисунок 7 – Отклонение выдачи сверх остатка

Пример выдачи с предупреждением. Остаток скрепок KT-005 – 16 уп., минимальный остаток – 3 уп. Выдача 13 уп. допустима ($13 \le 16$), новый остаток $16 - 13 = 3$, и так как $3 \le 3$, пользователь видит предупреждение о необходимости пополнения (рисунок 8).

Контроллер только передаёт форму в сервис и выбирает, какую страницу показать (листинг 5). Проверок и обращений к базе в нём нет – это требование разделения слоёв.

Листинг 5 – Обработка формы выдачи (Controllers/OperationsController.cs)

```
[HttpPost]
[ValidateAntiForgeryToken]   // защита от подделки запроса с чужого сайта
public IActionResult Issue(IssueForm form)
{
    if (ModelState.IsValid)                          // проверки атрибутов формы
    {
        var result = _stockService.Issue(form);   // вся логика — в сервисе
        if (result.Success)
        {
            TempData["Message"] = $"Выдача оформлена: {form.Quantity} ед.";
            if (result.Warning != null)
                TempData["Warning"] = result.Warning;
            return RedirectToAction("Details", "Products", new { id =
form.ProductId });
        }
        ModelState.AddModelError(string.Empty, result.Error!);   // текст
ошибки на форму
    }
    FillLists(onlyActiveProducts: true);
    return View(form);                               // та же форма с введёнными
данными
}
```

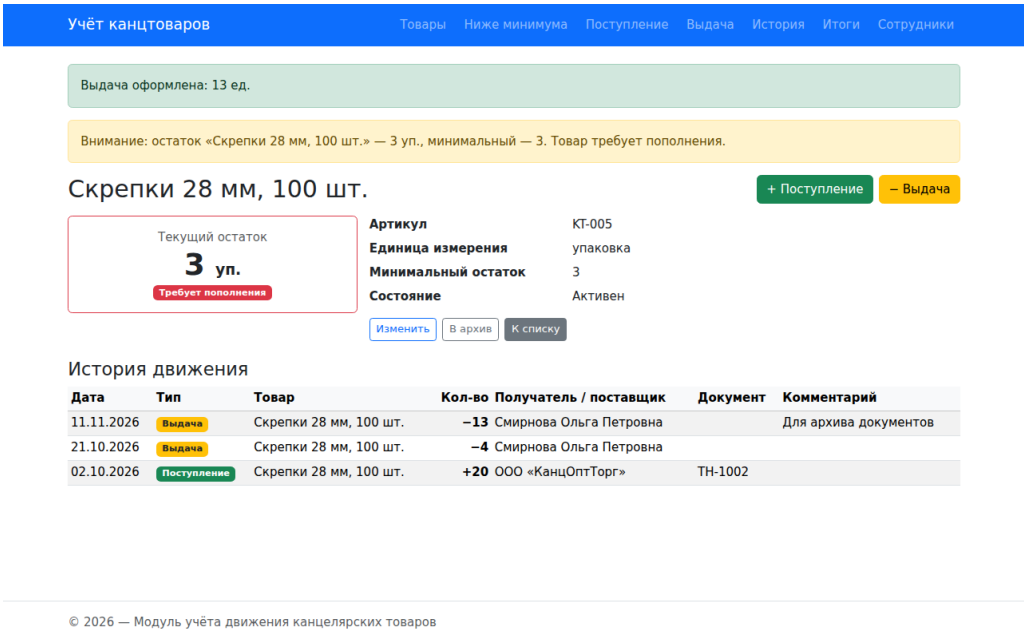


Рисунок 8 – Выдача с предупреждением о необходимости пополнения

Для поступления сервис проверяет, что указан поставщик или номер документа; пустые строки и строки из пробелов не считаются заполненными. Общие проверки количества и даты вынесены в отдельный метод CheckQuantityAndDate, который используют обе операции.

3.3 Расчёт остатка и контроль минимума

Формулы (1) и (3) реализованы в статическом классе StockCalculator (листинг 6).

Листинг 6 – Расчёт остатка и контроль минимума (Services/StockCalculator.cs)

```
public static int CalculateBalance(IEnumerable<StockOperation> operations)
{
    int received = operations.Where(o => o.Type ==
OperationType.Receipt).Sum(o => o.Quantity);
    int issued   = operations.Where(o => o.Type == OperationType.Issue).Sum(o
=> o.Quantity);
    return received - issued;
}

public static bool NeedsRestock(int balance, int minQuantity) => balance <=
minQuantity;
```

Пример расчёта для ручки КТ-001 по учебным данным: поступило 100 шт., выдано 30 + 25 + 30 = 85 шт., остаток О = 100 − 85 = 15 шт. Минимальный остаток – 20 шт., 15 ≤ 20, поэтому товар отмечен «Требует пополнения». Класс не обращается к базе данных, поэтому его функции проверяются тестами без

подготовки базы. Остаток и состояние товара показаны в карточке товара, на которой также выводится история движения именно этого товара (рисунок 9). Отдельная страница «Ниже минимума» показывает только активные товары, требующие пополнения.

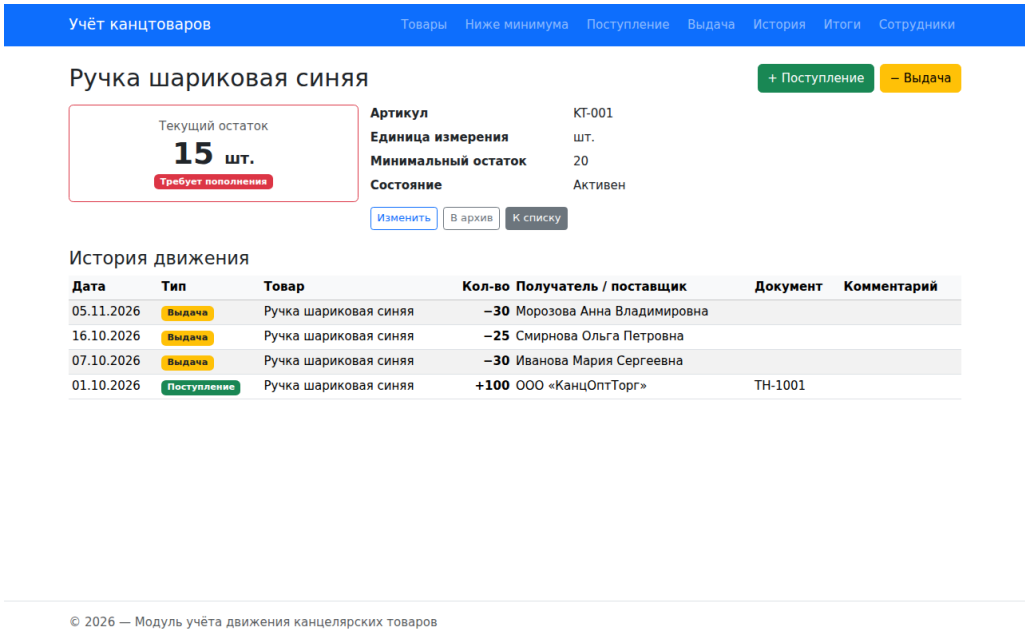


Рисунок 9 – Карточка товара: остаток и история движения

3.4 История движения и итоги за период

История движения реализована методом StockService.GetHistory (листинг 7). Условия фильтра добавляются к запросу по одному: запрос LINQ формируется как IQueryable и выполняется в базе только при вызове ToList(), поэтому из базы читаются только подходящие записи.

Листинг 7 – Фильтрация истории движения (Services/StockService.cs, метод GetHistory)

```
IQueryable<StockOperation> query = _db.StockOperations
    .AsNoTracking()
    .Include(o => o.Product)
    .Include(o => o.Employee);

if (filter.ProductId != null)  query = query.Where(o => o.ProductId ==
filter.ProductId);
if (filter.Type != null)       query = query.Where(o => o.Type ==
filter.Type);
if (filter.EmployeeId != null) query = query.Where(o => o.EmployeeId ==
filter.EmployeeId);
if (filter.From != null)       query = query.Where(o => o.Date >=
filter.From.Value.Date);
if (filter.To != null)         query = query.Where(o => o.Date <=
filter.To.Value.Date);

return query.OrderByDescending(o => o.Date).ThenByDescending(o =>
o.Id).ToList();
```

Пример: при фильтре «товар КТ-001, тип „Выдача“» в запрос попадают два условия Where, и база возвращает три выдачи ручки на сумму 85 шт. Пользователь может отобрать операции по товару, типу, сотруднику и периоду; над таблицей выводятся суммы поступлений и выдач по найденным операциям (рисунок 10).

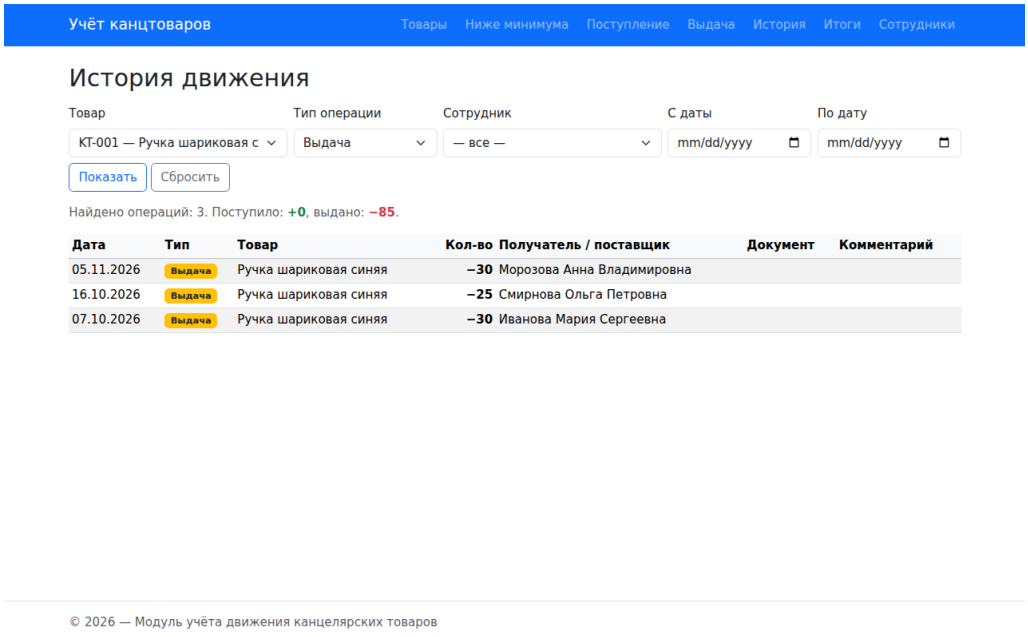


Рисунок 10 – История движения с фильтром по товару и типу операции

Итоги за период (рисунок 11) группируют операции по товару и показывают, сколько каждого товара поступило и выдано. Если дата начала

периода позже даты окончания, вместо таблицы выводится сообщение об ошибке.

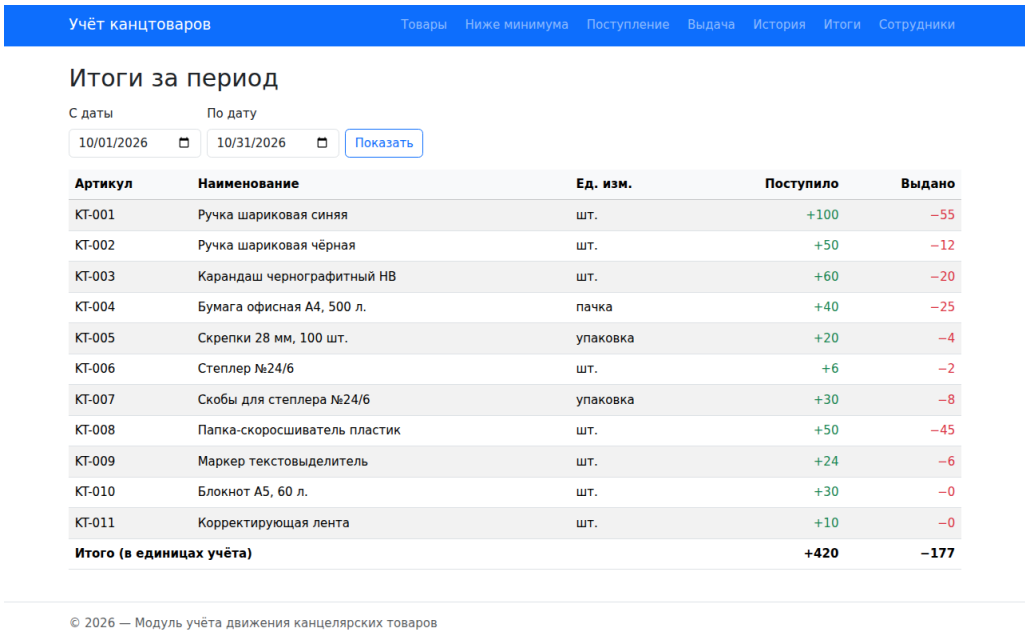


Рисунок 11 – Итоги поступлений и выдач за октябрь 2026 года

3.5 Хранение данных и развёртывание

База SQLite создаётся миграцией InitialCreate. При запуске приложение вызывает метод Database.Migrate(), который создаёт файл stationery.db и таблицы, если их ещё нет, а затем DbSeeder добавляет учебные данные: 12 товаров, 8 сотрудников и 27 операций за октябрь – ноябрь 2026 года. Если товары в базе уже есть, учебные данные повторно не добавляются, поэтому изменения пользователя сохраняются между запусками. Чтобы вернуть учебную базу в исходное состояние, достаточно остановить приложение и удалить файл stationery.db.

Для развёртывания нужен только .NET SDK 10: пакеты NuGet скачиваются автоматически при первой сборке. Порядок запуска приведён в приложении Д. Исходный код хранится в репозитории Git; каждый этап работы зафиксирован отдельными коммитами. После каждой отправки изменений на GitHub автоматически выполняются сборка и все тесты (GitHub Actions).

3.6 Тестирование программного модуля

Подход к тестированию. Автоматическими тестами проверяется предметная логика, которую можно проверить отдельно от интерфейса: расчёт остатка, главное правило выдачи, контроль минимума, проверки ввода, поиск, фильтры, итоги и сохранение данных. Тесты вызывают методы сервисов напрямую и используют базу SQLite в оперативной памяти: для каждого теста создаётся новая пустая база, поэтому тесты не зависят друг от друга. Работа интерфейса, сохранение после перезапуска приложения и развёртывание проверены вручную через браузер и HTTP-запросы.

Каждый тест построен по схеме «подготовка – действие – проверка». В листинге 8 приведён тест граничного случая главного правила: при остатке 10 шт. выдача 11 шт. должна быть отклонена, а остаток не должен измениться.

Листинг 8 – Тест граничного случая «остаток + 1» (StationeryAccounting.Tests/IssueTests.cs)

```
[Fact]
public void ВыдатьТовар_ОстатокПлюсОдин_Отклоняется()
{
    // Подготовка (в конструкторе класса): ручка, поступило 10 шт., минимум 3
    // Действие:
    var result = Issue(11);

    // Проверка: выдача отклонена, остаток прежний
    Assert.False(result.Success);
    Assert.Equal(10, Balance());
}
```

Тестовый проект StationeryAccounting.Tests содержит 36 тестовых методов; с учётом параметризованных тестов (Theory) выполняется 48 проверок. Состав тестов приведён в таблице 7, полный перечень с исходными данными и ожидаемыми результатами – в приложении Б.

Таблица 7 – Состав автоматических тестов

Тестовый класс	Функции	Проверок	Что проверяется	Результат
StockCalculatorTests	Ф4, Ф6	7	формула остатка; признак «Требует пополнения» при остатке меньше, равном и больше минимума	пройдены
IssueTests	Ф3, Ф6	13	главное правило; выдача ровно всего остатка и «остаток + 1»; пустой получатель, количество 0 и −5, дата в будущем, архивный товар, неизвестные Id; предупреждение о минимуме	пройдены
ReceiptTests	Ф2	6	поступление увеличивает остаток; обязательность поставщика или документа; архивный товар; количество 0; неизвестный товар	пройдены
ProductServiceTests	Ф1, Ф4, Ф6	13	добавление с нормализацией, повтор артикула, пустое наименование, минимум −1 и 0, неизвестный Id, повторное архивирование, поиск, фильтры	пройдены
HistoryAndTotalsTests	Ф5, Ф7	6	история без фильтра, по товару и типу, по сотруднику, по периоду с включёнными границами; итоги за период и за период без операций	пройдены
EmployeeAndPersistenceTests	Ф8, сохранение	3	пустое ФИО, неизвестный Id; запись в одном DbContext и чтение в новом	пройдены

Тесты запущены командой dotnet test на сервере GitHub Actions (Ubuntu, .NET 10). Результат: всего 48 проверок, пройдено 48, не пройдено 0 (рисунок 12). Полный код тестовых классов приведён в приложении А.

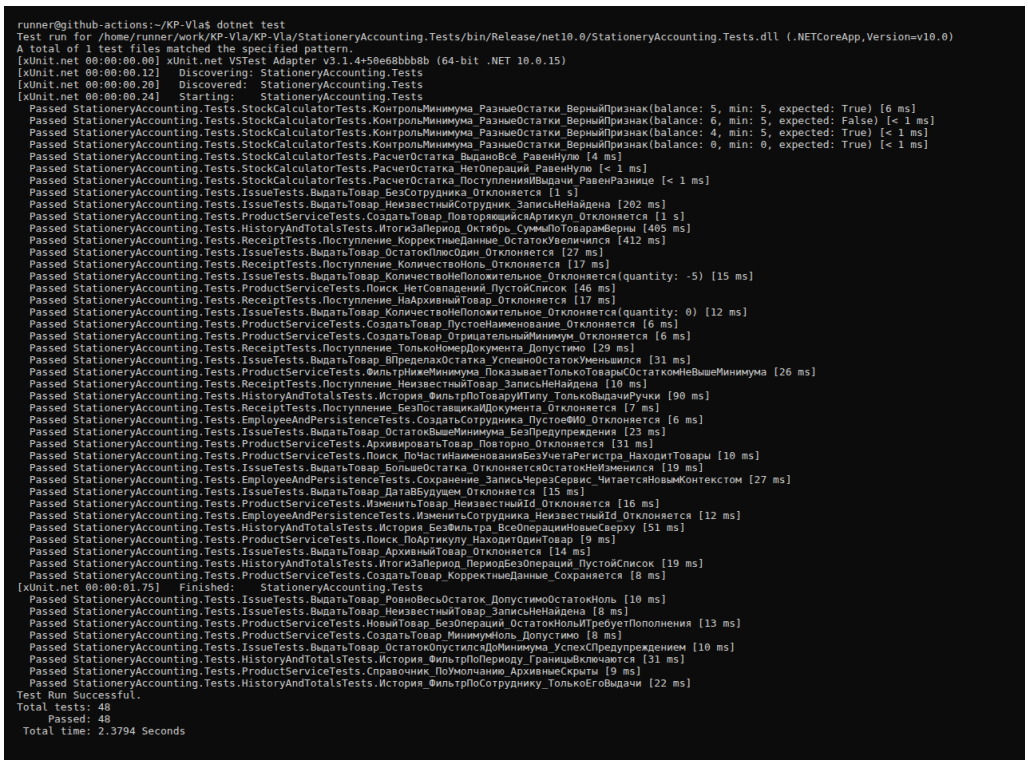


Рисунок 12 – Результаты выполнения автоматических тестов

Ручные проверки. Сценарии, связанные с интерфейсом и перезапуском приложения, проверены в браузере. Проверки выполнялись автоматизированным браузером (Google Chrome под управлением Playwright) по сценарию, записанному в файле tools/screenshots.py: сценарий выполнял действия пользователя, читал со страницы фактически появившееся сообщение и сохранял снимок экрана. Фактические результаты – это тексты, прочитанные со страниц, а не ожидаемые значения. Всего выполнено 25 ручных проверок; основные из них приведены в таблице 8, полная таблица – в приложении В.

Таблица 8 – Основные ручные проверки (выборка)

№	Функция	Исходные данные и действия	Ожидаемый результат	Фактический результат	Вывод
1	Ф1 Список	открыть «Товары»	11 активных товаров, 4 выделены	Отображено товаров: 11, выделено «Требует пополнения»: 4	выполнено
4	Ф1 Повтор артикула	добавить товар с артикулом «kt-001»	отказ	Товар с артикулом «КТ-001» уже существует	выполнено
8	Ф2 Поступление	КТ-001 +50, поставщик и ТН-1100	остаток 15 → 65	Поступление оформлено: 50 ед.; остаток КТ-001 стал: 65 шт.	выполнено
10	Ф3 Сверх остатка	КТ-002 (остаток 38), выдать 100	отказ	Недостаточно товара «Ручка шариковая чёрная»: на складе 38 шт., запрошено 100 шт.	выполнено
11	Ф3 Без получателя	КТ-002, выдать 1, сотрудник не выбран	отказ	Для выдачи укажите сотрудника-получателя	выполнено
14	Ф3, Ф6 До минимума	КТ-005 (16 уп., минимум 3), выдать 13	успех с предупреждением	Выдача оформлена: 13 ед. / Внимание: остаток «Скрепки 28 мм, 100 шт.» — 3 уп., минимальный — 3. Товар требует пополнения.; остаток КТ-005: 3 уп.	выполнено
15	Ф3 Весь остаток	КТ-006 (4 шт.), выдать 4	успех, остаток 0	Выдача оформлена: 4 ед. / Внимание: остаток «Степлер №24/6» — 0 шт., минимальный — 2. Товар требует пополнения.; остаток КТ-006: 0 шт.	выполнено
18	Ф5 Фильтр истории	товар КТ-001, тип «Выдача»	3 выдачи, −85	Найдено операций: 3. Поступило: +0, выдано: −85.	выполнено
21	Ф7 Итоги	октябрь 2026	суммы по товарам	Итого (в единицах учёта) +420 −177	выполнено
24	Неизвестный Id	/Products/Details/999 и /api/products/999	404 без падения	HTTP 404: Запись не найдена (404); HTTP 404: Товар с Id = 999 не найден	выполнено
25	Сохранение после перезапуска	остановить и снова запустить приложение, найти КТ-013, открыть КТ-001	данные на месте	КТ-013 найден: да (Скотч прозрачный 19 мм); остаток КТ-001 после перезапуска: 65 шт.	выполнено

Развёртывание из архива проверено так: архив проекта, созданный командой git archive, распакован в новую папку, после чего выполнены шаги инструкции запуска (приложение Д) – сборка, автоматическое создание базы, открытие страниц. На сервере GitHub Actions та же последовательность выполняется при каждом запуске проверок на чистой машине без заранее созданной базы.

Найденные ошибки и их исправления приведены в таблице 9.

Таблица 9 – Найденные ошибки и исправления

№	Ошибка	Причина	Исправление	Повторная проверка
1	В предупреждении выводилось «3 упаковка»	единица измерения подставлялась в полной форме без согласования с числом	добавлены сокращения единиц (Units.Short): «3 уп.»	проверка 14 повторена: «3 уп.»
2	Ошибка проверки данных в API выводилась с английским заголовком «One or more validation errors occurred»	использовался стандартный ответ ASP.NET Core	задан собственный ответ { error } с текстами на русском	проверка 13 повторена: ответ на русском
3	Первый запуск не создавал таблицы	отсутствовала миграция при вызове Database.Migrate()	создана миграция InitialCreate	база создаётся при каждом чистом запуске

Оставшиеся ограничения: в модуле нет разграничения прав и входа в систему (не предусмотрено заданием); поиск выполняется в памяти, что подходит для справочника до нескольких тысяч позиций; защита от одновременной выдачи действует в пределах одного экземпляра приложения; стили Bootstrap подключаются из сети, поэтому без интернета страницы открываются без оформления.

Выводы по тестированию. Все 48 автоматических проверок пройдены, все 25 ручных проверок дали ожидаемый результат. Главное правило и все граничные случаи – выдача ровно всего остатка, «остаток + 1», остаток, равный минимуму, новый товар без операций – подтверждены тестами. Найденные в ходе проверок ошибки исправлены, проверки повторены.

Выводы по разделу 3

Реализованы все восемь обязательных функций: справочник товаров с поиском и архивом, поступление и выдача, автоматический расчёт остатка, история движения с фильтрами, контроль минимума, итоги за период и справочник сотрудников. Предметная логика сосредоточена в сервисах и проверена 48 автоматическими тестами; интерфейс и сохранение данных после перезапуска проверены вручную. Модуль разворачивается одной командой, база создаётся автоматически.

ЗАКЛЮЧЕНИЕ

В ходе курсового проектирования разработан модуль учёта движения канцелярских товаров. Поставленная во введении цель достигнута, что подтверждается результатами решения каждой задачи.

Первая задача – анализ предметной области – решена в разделе 1: описан процесс учёта, выявлены проблемы ручного ведения (расхождение остатка, выдача сверх наличия, отсутствие истории), рассмотрены аналоги «1С:Бухгалтерия», «МойСклад» и электронные таблицы. Сформулированы восемь функциональных требований и ограничения предметной области.

Вторая задача – выбор средств разработки – решена в подразделе 1.4: выбраны C# и .NET 10, ASP.NET Core MVC, Entity Framework Core с базой SQLite, xUnit и Git. Выбор обоснован характером пользователя (нужны страницы, а не программный интерфейс) и масштабом задачи (одна база в файле без отдельного сервера).

Третья задача – проектирование – решена в разделе 2: построена модель данных из трёх сущностей, остаток определён формулой «сумма поступлений минус сумма выдач» и не хранится в базе, выбрана трёхслойная архитектура, построены диаграмма вариантов использования и блок-схема алгоритма выдачи.

Четвёртая задача – реализация – решена в подразделах 3.1–3.5: все функции задания работают, неверные данные отклоняются с понятными сообщениями на русском языке, обращение к несуществующей записи даёт страницу «Запись не найдена», одновременные выдачи одного товара выполняются по очереди.

Пятая задача – тестирование – решена в подразделе 3.6: 48 автоматических проверок и 25 ручных проверок пройдены, три найденные ошибки исправлены и проверены повторно.

Шестая задача – подготовка документации – решена: составлены инструкция запуска и инструкция пользователя (приложения Д и Е).

Перспективы развития модуля: вход в систему с ролями «кладовщик» и «руководитель»; учёт стоимости товаров и сумм поступлений; формирование

заказа поставщику по товарам ниже минимума; выгрузка истории и итогов в Excel; инвентаризация с автоматическим созданием корректирующих операций.

«___» _____ 2026 г. _____ / _____

(подпись) (ФИО)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Законы. О бухгалтерском учете : Федеральный закон № 402-ФЗ : [принят Государственной думой 22 ноября 2011 года : одобрен Советом Федерации 29 ноября 2011 года]. – Текст : электронный // КонсультантПлюс. – URL: https://www.consultant.ru/document/cons_doc_LAW_122855/ (дата обращения: 11.11.2026).

2. Об утверждении Федерального стандарта бухгалтерского учета ФСБУ 5/2019 «Запасы» : Приказ Министерства финансов Российской Федерации от 15 ноября 2019 г. № 180н. – Текст : электронный // КонсультантПлюс. – URL: https://www.consultant.ru/document/cons_doc_LAW_339442/ (дата обращения: 11.11.2026).

3. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения : издание официальное : утвержден и введен в действие Постановлением Государственного комитета СССР по управлению качеством продукции и стандартам от 26.12.90 № 3294 : дата введения 1992-01-01. – Москва : Стандартинформ, 2010. – 22 с.

4. Лок, Э. ASP.NET Core в действии / Э. Лок ; перевод с английского Д. А. Беликова. – Москва : ДМК Пресс, 2021. – 906 с. – ISBN 978-5-97060-550-9. [проверить]

5. Прайс, М. C# 10 и .NET 6. Современная кросс-платформенная разработка / М. Прайс. – Санкт-Петербург : Питер, 2023. – 848 с. – ISBN 978-5-4461-2249-0. [проверить]

6. Смит, Д. Entity Framework Core в действии / Д. Смит ; перевод с английского Д. А. Беликова. – Москва : ДМК Пресс, 2022. – 690 с. – ISBN 978-5-93700-114-6. [проверить]

7. Хориков, В. Принципы юнит-тестирования / В. Хориков. – Санкт-Петербург : Питер, 2021. – 320 с. – ISBN 978-5-4461-1683-3.

8. Чакон, С. Git для профессионального программиста / С. Чакон, Б. Штрауб. – Санкт-Петербург : Питер, 2016. – 496 с. – ISBN 978-5-496-01763-3.

9. Lock, A. ASP.NET Core in Action / A. Lock. – 3rd ed. – Shelter Island : Manning, 2023. – 984 p. – ISBN 978-1-63343-862-3. [проверить]

10. Overview of ASP.NET Core MVC. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/aspnet/core/mvc/overview (дата обращения: 11.11.2026).

11. Entity Framework Core. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/ef/core/ (дата обращения: 11.11.2026).

12. Unit testing C# with xUnit and .NET. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/dotnet/core/testing/unit-testing-csharp-with-xunit (дата обращения: 11.11.2026).

13. SQLite Documentation. – Текст : электронный // SQLite. – URL: https://www.sqlite.org/docs.html (дата обращения: 11.11.2026).

Приложение А

Листинг автоматических тестов

В приложении приведён полный код тестового проекта StationeryAccounting.Tests (раздел 3.6).

Файл TestDb.cs

```
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using StationeryAccounting.Data;
using StationeryAccounting.Models;

namespace StationeryAccounting.Tests;

/// <summary>
/// Тестовая база SQLite в оперативной памяти. Для каждого теста — своя чистая
база.
/// Соединение держим открытым: пока оно открыто, база в памяти существует,
/// и разные DbContext видят одни и те же данные (так проверяем сохранение).
/// </summary>
public sealed class TestDb : IDisposable
{
    private readonly SqliteConnection _connection;
    private readonly DbContextOptions<AppDbContext> _options;

    public TestDb()
    {
        _connection = new SqliteConnection("Data Source=:memory:");
        _connection.Open();
        _options = new DbContextOptionsBuilder<AppDbContext>()
            .UseSqlite(_connection)
            .Options;

        using var db = CreateContext();
        db.Database.EnsureCreated();   // создаём таблицы по модели (без учебных
данных)
    }

    /// <summary>Новый контекст к той же базе.</summary>
    public AppDbContext CreateContext() => new(_options);

    public Product AddProduct(string article, string name, int minQuantity = 5,
bool archived = false, string unit = "шт.")
    {
        using var db = CreateContext();
        var product = new Product { Article = article, Name = name, MinQuantity
= minQuantity, IsArchived = archived, Unit = unit };
        db.Products.Add(product);
        db.SaveChanges();
        return product;
    }

    public Employee AddEmployee(string fullName = "Иванова Мария Сергеевна",
string department = "Бухгалтерия")
    {
        using var db = CreateContext();
        var employee = new Employee { FullName = fullName, Department =
department };
        db.Employees.Add(employee);
```

```
        db.SaveChanges();
        return employee;
    }

    /// <summary>Добавить операцию напрямую в базу (подготовка исходных данных
теста).</summary>
    public void AddOperation(int productId, OperationType type, int quantity,
DateTime date, int? employeeId = null)
    {
        using var db = CreateContext();
        db.StockOperations.Add(new StockOperation
        {
            ProductId = productId,
            Type = type,
            Quantity = quantity,
            Date = date,
            EmployeeId = employeeId,
            Supplier = type == OperationType.Receipt ? "ООО «Тест»" : null,
            CreatedAt = date
        });
        db.SaveChanges();
    }

    public void Dispose() => _connection.Dispose();
}
```

Файл StockCalculatorTests.cs

```
using StationeryAccounting.Models;
using StationeryAccounting.Services;

namespace StationeryAccounting.Tests;

/// <summary>Расчёт остатка и контроль минимума — без базы данных.</summary>
public class StockCalculatorTests
{
    private static StockOperation Op(OperationType type, int quantity) =>
        new() { Type = type, Quantity = quantity };

    [Fact]
    public void РасчетОстатка_ПоступленияИВыдачи_РавенРазнице()
    {
        // 100 + 20 поступило, 30 + 25 выдано → 65
        var ops = new[]
        {
            Op(OperationType.Receipt, 100), Op(OperationType.Receipt, 20),
            Op(OperationType.Issue, 30), Op(OperationType.Issue, 25)
        };

        Assert.Equal(65, StockCalculator.CalculateBalance(ops));
    }

    [Fact]
    public void РасчетОстатка_НетОпераций_РавенНулю()
    {
        Assert.Equal(0,
StockCalculator.CalculateBalance(Array.Empty<StockOperation>()));
    }

    [Fact]
    public void РасчетОстатка_ВыданоВсё_РавенНулю()
    {
        var ops = new[] { Op(OperationType.Receipt, 10), Op(OperationType.Issue,
10) };
```

```
        Assert.Equal(0, StockCalculator.CalculateBalance(ops));
    }

    [Theory]
    [InlineData(4, 5, true)]    // меньше минимума
    [InlineData(5, 5, true)]    // ровно минимум — уже «Требует пополнения»
    [InlineData(6, 5, false)]   // больше минимума
    [InlineData(0, 0, true)]    // минимум 0, остатка нет
    public void КонтрольМинимума_РазныеОстатки_ВерныйПризнак(int balance, int
min, bool expected)
    {
        Assert.Equal(expected, StockCalculator.NeedsRestock(balance, min));
    }
}
```

Файл IssueTests.cs

```
using StationeryAccounting.Models;
using StationeryAccounting.Services;
using StationeryAccounting.ViewModels;

namespace StationeryAccounting.Tests;

/// <summary>
/// Выдача товара — главное правило модуля: нельзя выдать больше, чем есть на
складе.
/// Исходные данные во всех тестах: ручка, поступило 10 шт., минимум 3.
/// </summary>
public class IssueTests : IDisposable
{
    private readonly TestDb _testDb = new();
    private readonly Product _pen;
    private readonly Employee _employee;

    public IssueTests()
    {
        _pen = _testDb.AddProduct("КТ-001", "Ручка шариковая синяя",
minQuantity: 3);
        _employee = _testDb.AddEmployee();
        _testDb.AddOperation(_pen.Id, OperationType.Receipt, 10,
DateTime.Today.AddDays(-5));
    }

    public void Dispose() => _testDb.Dispose();

    private OperationResult Issue(int quantity, int? employeeId = -1, DateTime?
date = null, int? productId = null)
    {
        using var db = _testDb.CreateContext();
        var service = new StockService(db);
        return service.Issue(new IssueForm
        {
            ProductId = productId ?? _pen.Id,
            Quantity = quantity,
            EmployeeId = employeeId == -1 ? _employee.Id : employeeId,
            Date = date ?? DateTime.Today
        });
    }

    private int Balance()
    {
        using var db = _testDb.CreateContext();
        return new StockService(db).GetBalance(_pen.Id);
    }
```

```
    [Fact]
    public void ВыдатьТовар_ВПределахОстатка_УспешноОстатокУменьшился()
    {
        var result = Issue(4);

        Assert.True(result.Success);
        Assert.Equal(6, Balance());
    }

    [Fact]
    public void ВыдатьТовар_БольшеОстатка_ОтклоняетсяОстатокНеИзменился()
    {
        // Пример из задания: выдать 15 ручек при остатке 10
        var result = Issue(15);

        Assert.False(result.Success);
        Assert.Contains("Недостаточно товара", result.Error);
        Assert.Equal(10, Balance());
    }

    [Fact]
    public void ВыдатьТовар_РовноВесьОстаток_ДопустимоОстатокНоль()
    {
        var result = Issue(10);

        Assert.True(result.Success);
        Assert.Equal(0, Balance());
    }

    [Fact]
    public void ВыдатьТовар_ОстатокПлюсОдин_Отклоняется()
    {
        var result = Issue(11);

        Assert.False(result.Success);
        Assert.Equal(10, Balance());
    }

    [Fact]
    public void ВыдатьТовар_БезСотрудника_Отклоняется()
    {
        var result = Issue(2, employeeId: null);

        Assert.False(result.Success);
        Assert.Equal("Для выдачи укажите сотрудника-получателя", result.Error);
    }

    [Theory]
    [InlineData(0)]
    [InlineData(-5)]
    public void ВыдатьТовар_КоличествоНеПоложительное_Отклоняется(int quantity)
    {
        var result = Issue(quantity);

        Assert.False(result.Success);
        Assert.Equal("Количество должно быть целым числом больше 0",
result.Error);
    }

    [Fact]
    public void ВыдатьТовар_ДатаВБудущем_Отклоняется()
    {
        var result = Issue(1, date: DateTime.Today.AddDays(1));
```

```
        Assert.False(result.Success);
        Assert.Equal("Дата операции не может быть в будущем", result.Error);
    }

    [Fact]
    public void ВыдатьТовар_АрхивныйТовар_Отклоняется()
    {
        var archived = _testDb.AddProduct("КТ-099", "Старый маркер", archived:
true);

        var result = Issue(1, productId: archived.Id);

        Assert.False(result.Success);
        Assert.Contains("архивным", result.Error);
    }

    [Fact]
    public void ВыдатьТовар_НеизвестныйТовар_ЗаписьНеНайдена()
    {
        var result = Issue(1, productId: 999);

        Assert.False(result.Success);
        Assert.True(result.IsNotFound);
    }

    [Fact]
    public void ВыдатьТовар_НеизвестныйСотрудник_ЗаписьНеНайдена()
    {
        var result = Issue(1, employeeId: 999);

        Assert.False(result.Success);
        Assert.True(result.IsNotFound);
    }

    [Fact]
    public void ВыдатьТовар_ОстатокОпустилсяДоМинимума_УспехСПредупреждением()
    {
        // 10 − 7 = 3, минимум 3 → «Требует пополнения», но выдача разрешена
        var result = Issue(7);

        Assert.True(result.Success);
        Assert.NotNull(result.Warning);
        Assert.Contains("требует пополнения", result.Warning);
    }

    [Fact]
    public void ВыдатьТовар_ОстатокВышеМинимума_БезПредупреждения()
    {
        // 10 − 6 = 4 > 3
        var result = Issue(6);

        Assert.True(result.Success);
        Assert.Null(result.Warning);
    }
}
```

Файл ReceiptTests.cs

```
using StationeryAccounting.Services;
using StationeryAccounting.ViewModels;

namespace StationeryAccounting.Tests;

/// <summary>Поступление товара на склад.</summary>
public class ReceiptTests : IDisposable
```

```
{
    private readonly TestDb _testDb = new();

    public void Dispose() => _testDb.Dispose();

    private OperationResult Receive(ReceiptForm form)
    {
        using var db = _testDb.CreateContext();
        return new StockService(db).Receive(form);
    }

    [Fact]
    public void Поступление_КорректныеДанные_ОстатокУвеличился()
    {
        var paper = _testDb.AddProduct("КТ-004", "Бумага А4");

        var result = Receive(new ReceiptForm
        {
            ProductId = paper.Id, Quantity = 40, Date = DateTime.Today, Supplier
= "ООО «Бумажный мир»"
        });

        using var db = _testDb.CreateContext();
        Assert.True(result.Success);
        Assert.Equal(40, new StockService(db).GetBalance(paper.Id));
    }

    [Fact]
    public void Поступление_БезПоставщикаИДокумента_Отклоняется()
    {
        var paper = _testDb.AddProduct("КТ-004", "Бумага А4");

        var result = Receive(new ReceiptForm { ProductId = paper.Id, Quantity =
5, Date = DateTime.Today, Supplier = "   " });

        Assert.False(result.Success);
        Assert.Equal("Для поступления укажите поставщика или номер документа",
result.Error);
    }

    [Fact]
    public void Поступление_ТолькоНомерДокумента_Допустимо()
    {
        var paper = _testDb.AddProduct("КТ-004", "Бумага А4");

        var result = Receive(new ReceiptForm { ProductId = paper.Id, Quantity =
5, Date = DateTime.Today, DocumentNumber = "ТН-77" });

        Assert.True(result.Success);
    }

    [Fact]
    public void Поступление_НаАрхивныйТовар_Отклоняется()
    {
        var archived = _testDb.AddProduct("КТ-011", "Корректирующая лента",
archived: true);

        var result = Receive(new ReceiptForm { ProductId = archived.Id, Quantity
= 5, Date = DateTime.Today, Supplier = "ООО «Тест»" });

        Assert.False(result.Success);
        Assert.Contains("архивным", result.Error);
    }
```

```
    [Fact]
    public void Поступление_КоличествоНоль_Отклоняется()
    {
        var paper = _testDb.AddProduct("КТ-004", "Бумага А4");

        var result = Receive(new ReceiptForm { ProductId = paper.Id, Quantity =
0, Date = DateTime.Today, Supplier = "ООО «Тест»" });

        Assert.False(result.Success);
    }

    [Fact]
    public void Поступление_НеизвестныйТовар_ЗаписьНеНайдена()
    {
        var result = Receive(new ReceiptForm { ProductId = 999, Quantity = 1,
Date = DateTime.Today, Supplier = "ООО «Тест»" });

        Assert.True(result.IsNotFound);
    }
}
```

Файл ProductServiceTests.cs

```
using StationeryAccounting.Models;
using StationeryAccounting.Services;
using StationeryAccounting.ViewModels;

namespace StationeryAccounting.Tests;

/// <summary>Справочник товаров: добавление, изменение, архив, поиск и
фильтры.</summary>
public class ProductServiceTests : IDisposable
{
    private readonly TestDb _testDb = new();

    public void Dispose() => _testDb.Dispose();

    private ProductService Service(out Data.AppDbContext db)
    {
        db = _testDb.CreateContext();
        return new ProductService(db);
    }

    [Fact]
    public void СоздатьТовар_КорректныеДанные_Сохраняется()
    {
        var service = Service(out var db);

        var result = service.Create(new Product { Article = "kt-020", Name = "
Ластик  ", Unit = "шт.", MinQuantity = 3 });

        Assert.True(result.Success);
        var saved = Assert.Single(service.GetAll());
        Assert.Equal("КТ-020", saved.Article);   // артикул приведён к верхнему
регистру
        Assert.Equal("Ластик", saved.Name);      // пробелы по краям убраны
        db.Dispose();
    }

    [Fact]
    public void СоздатьТовар_ПовторяющийсяАртикул_Отклоняется()
    {
        _testDb.AddProduct("КТ-001", "Ручка синяя");
        var service = Service(out var db);
```

```
        // Тот же артикул в другом регистре тоже считается повтором
        var result = service.Create(new Product { Article = "kt-001", Name =
"Ручка красная", Unit = "шт." });

        Assert.False(result.Success);
        Assert.Contains("уже существует", result.Error);
        db.Dispose();
    }

    [Fact]
    public void СоздатьТовар_ПустоеНаименование_Отклоняется()
    {
        var service = Service(out var db);

        var result = service.Create(new Product { Article = "КТ-030", Name = "
", Unit = "шт." });

        Assert.False(result.Success);
        Assert.Equal("Укажите наименование", result.Error);
        db.Dispose();
    }

    [Fact]
    public void СоздатьТовар_ОтрицательныйМинимум_Отклоняется()
    {
        var service = Service(out var db);

        var result = service.Create(new Product { Article = "КТ-031", Name =
"Скотч", Unit = "шт.", MinQuantity = -1 });

        Assert.False(result.Success);
        Assert.Equal("Минимальный остаток не может быть отрицательным",
result.Error);
        db.Dispose();
    }

    [Fact]
    public void СоздатьТовар_МинимумНоль_Допустимо()
    {
        var service = Service(out var db);

        var result = service.Create(new Product { Article = "КТ-032", Name =
"Скотч", Unit = "шт.", MinQuantity = 0 });

        Assert.True(result.Success);
        db.Dispose();
    }

    [Fact]
    public void ИзменитьТовар_НеизвестныйId_Отклоняется()
    {
        var service = Service(out var db);

        var result = service.Update(new Product { Id = 999, Article = "КТ-001",
Name = "Ручка", Unit = "шт." });

        Assert.False(result.Success);
        Assert.Equal("Товар не найден", result.Error);
        Assert.Null(service.GetById(999));
        db.Dispose();
    }

    [Fact]
    public void АрхивироватьТовар_Повторно_Отклоняется()
```

```
{
    var pen = _testDb.AddProduct("КТ-001", "Ручка синяя");
    var service = Service(out var db);

    var first = service.Archive(pen.Id);
    var second = service.Archive(pen.Id);

    Assert.True(first.Success);
    Assert.False(second.Success);
    Assert.True(service.GetById(pen.Id)!.IsArchived);
    db.Dispose();
}

[Fact]
public void НовыйТовар_БезОпераций_ОстатокНольИТребуетПополнения()
{
    _testDb.AddProduct("КТ-012", "Клей-карандаш", minQuantity: 5);
    var service = Service(out var db);

    var item = Assert.Single(service.Search(new ProductFilter()));

    Assert.Equal(0, item.Balance);
    Assert.True(item.NeedsRestock);
    db.Dispose();
}

[Fact]
public void Поиск_ПоЧастиНаименованияБезУчетаРегистра_НаходитТовары()
{
    _testDb.AddProduct("КТ-001", "Ручка шариковая синяя");
    _testDb.AddProduct("КТ-002", "Ручка шариковая чёрная");
    _testDb.AddProduct("КТ-003", "Карандаш НВ");
    var service = Service(out var db);

    var found = service.Search(new ProductFilter { Search = "РУЧКА" });

    Assert.Equal(2, found.Count);
    db.Dispose();
}

[Fact]
public void Поиск_ПоАртикулу_НаходитОдинТовар()
{
    _testDb.AddProduct("КТ-001", "Ручка шариковая синяя");
    _testDb.AddProduct("КТ-003", "Карандаш НВ");
    var service = Service(out var db);

    var found = service.Search(new ProductFilter { Search = "кт-003" });

    Assert.Equal("Карандаш НВ", Assert.Single(found).Name);
    db.Dispose();
}

[Fact]
public void Поиск_НетСовпадений_ПустойСписок()
{
    _testDb.AddProduct("КТ-001", "Ручка шариковая синяя");
    var service = Service(out var db);

    Assert.Empty(service.Search(new ProductFilter { Search = "степлер" }));
    db.Dispose();
}

[Fact]
```

```
    public void
ФильтрНижеМинимума_ПоказываетТолькоТоварыСОстаткомНеВышеМинимума()
    {
        var low = _testDb.AddProduct("КТ-001", "Ручка", minQuantity: 5);      //
остаток 5 = минимум
        var ok = _testDb.AddProduct("КТ-002", "Карандаш", minQuantity: 5);    //
остаток 20
        var archived = _testDb.AddProduct("КТ-003", "Лента", minQuantity: 5,
archived: true);
        _testDb.AddOperation(low.Id, OperationType.Receipt, 5, DateTime.Today);
        _testDb.AddOperation(ok.Id, OperationType.Receipt, 20, DateTime.Today);
        var service = Service(out var db);

        var found = service.Search(new ProductFilter { BelowMinOnly = true,
ShowArchived = true });

        Assert.Equal(low.Id, Assert.Single(found).Id);   // архивный не попадает
в «Ниже минимума»
        db.Dispose();
    }

    [Fact]
    public void Справочник_ПоУмолчанию_АрхивныеСкрыты()
    {
        _testDb.AddProduct("КТ-001", "Ручка");
        _testDb.AddProduct("КТ-011", "Лента", archived: true);
        var service = Service(out var db);

        Assert.Single(service.Search(new ProductFilter()));
        Assert.Equal(2, service.Search(new ProductFilter { ShowArchived =
true }).Count);
        db.Dispose();
    }
}
```

Файл HistoryAndTotalsTests.cs

```
using StationeryAccounting.Models;
using StationeryAccounting.Services;
using StationeryAccounting.ViewModels;

namespace StationeryAccounting.Tests;

/// <summary>История движения с фильтрами и итоги за период.</summary>
public class HistoryAndTotalsTests : IDisposable
{
    private readonly TestDb _testDb = new();
    private readonly Product _pen;
    private readonly Product _paper;
    private readonly Employee _ivanova;
    private readonly Employee _petrov;

    // Исходные данные: октябрь 2026, по две операции на каждый товар
    public HistoryAndTotalsTests()
    {
        _pen = _testDb.AddProduct("КТ-001", "Ручка");
        _paper = _testDb.AddProduct("КТ-004", "Бумага А4");
        _ivanova = _testDb.AddEmployee("Иванова Мария Сергеевна");
        _petrov = _testDb.AddEmployee("Петров Алексей Викторович", "Отдел
продаж");

        _testDb.AddOperation(_pen.Id, OperationType.Receipt, 100, new
DateTime(2026, 10, 1));
        _testDb.AddOperation(_paper.Id, OperationType.Receipt, 40, new
DateTime(2026, 10, 2));
```

```
        _testDb.AddOperation(_pen.Id, OperationType.Issue, 30, new
DateTime(2026, 10, 10), _ivanova.Id);
        _testDb.AddOperation(_paper.Id, OperationType.Issue, 15, new
DateTime(2026, 10, 31), _petrov.Id);
    }

    public void Dispose() => _testDb.Dispose();

    private List<StockOperation> History(HistoryFilter filter)
    {
        using var db = _testDb.CreateContext();
        return new StockService(db).GetHistory(filter);
    }

    [Fact]
    public void История_БезФильтра_ВсеОперацииНовыеСверху()
    {
        var all = History(new HistoryFilter());

        Assert.Equal(4, all.Count);
        Assert.Equal(new DateTime(2026, 10, 31), all[0].Date);
    }

    [Fact]
    public void История_ФильтрПоТоваруИТипу_ТолькоВыдачиРучки()
    {
        var found = History(new HistoryFilter { ProductId = _pen.Id, Type =
OperationType.Issue });

        var op = Assert.Single(found);
        Assert.Equal(30, op.Quantity);
    }

    [Fact]
    public void История_ФильтрПоСотруднику_ТолькоЕгоВыдачи()
    {
        var found = History(new HistoryFilter { EmployeeId = _petrov.Id });

        Assert.Equal(_paper.Id, Assert.Single(found).ProductId);
    }

    [Fact]
    public void История_ФильтрПоПериоду_ГраницыВключаются()
    {
        // Период ровно с 2 по 10 октября — обе граничные даты должны попасть
        var found = History(new HistoryFilter { From = new DateTime(2026, 10,
2), To = new DateTime(2026, 10, 10) });

        Assert.Equal(2, found.Count);
    }

    [Fact]
    public void ИтогиЗаПериод_Октябрь_СуммыПоТоварамВерны()
    {
        using var db = _testDb.CreateContext();
        var rows = new StockService(db).GetTotals(new DateTime(2026, 10, 1), new
DateTime(2026, 10, 31));

        var pen = rows.Single(r => r.ProductId == _pen.Id);
        var paper = rows.Single(r => r.ProductId == _paper.Id);
        Assert.Equal((100, 30), (pen.Received, pen.Issued));
        Assert.Equal((40, 15), (paper.Received, paper.Issued));
    }
```

```
    [Fact]
    public void ИтогиЗаПериод_ПериодБезОпераций_ПустойСписок()
    {
        using var db = _testDb.CreateContext();

        var rows = new StockService(db).GetTotals(new DateTime(2026, 9, 1), new
DateTime(2026, 9, 30));

        Assert.Empty(rows);
    }
}
```

Файл EmployeeAndPersistenceTests.cs

```
using StationeryAccounting.Models;
using StationeryAccounting.Services;
using StationeryAccounting.ViewModels;

namespace StationeryAccounting.Tests;

/// <summary>Справочник сотрудников и сохранение данных в базе.</summary>
public class EmployeeAndPersistenceTests : IDisposable
{
    private readonly TestDb _testDb = new();

    public void Dispose() => _testDb.Dispose();

    [Fact]
    public void СоздатьСотрудника_ПустоеФИО_Отклоняется()
    {
        using var db = _testDb.CreateContext();

        var result = new EmployeeService(db).Create(new Employee { FullName =
"", Department = "ИТ-отдел" });

        Assert.False(result.Success);
        Assert.Equal("Укажите ФИО сотрудника", result.Error);
    }

    [Fact]
    public void ИзменитьСотрудника_НеизвестныйId_Отклоняется()
    {
        using var db = _testDb.CreateContext();

        var result = new EmployeeService(db).Update(new Employee { Id = 999,
FullName = "Кто-то", Department = "ИТ" });

        Assert.False(result.Success);
        Assert.Equal("Сотрудник не найден", result.Error);
    }

    [Fact]
    public void Сохранение_ЗаписьЧерезСервис_ЧитаетсяНовымКонтекстом()
    {
        // Записываем в одном контексте…
        using (var db = _testDb.CreateContext())
        {
            new ProductService(db).Create(new Product { Article = "КТ-050", Name
= "Дырокол", Unit = "шт.", MinQuantity = 1 });
        }
        int productId;
        using (var db = _testDb.CreateContext())
        {
            productId = new ProductService(db).GetAll().Single().Id;
```

```
            var employee = new Employee { FullName = "Смирнова Ольга Петровна",
Department = "Секретариат" };
            new EmployeeService(db).Create(employee);
            var stock = new StockService(db);
            stock.Receive(new ReceiptForm { ProductId = productId, Quantity = 3,
Date = DateTime.Today, Supplier = "ООО «ОфисСнаб»" });
            stock.Issue(new IssueForm { ProductId = productId, Quantity = 1,
Date = DateTime.Today, EmployeeId = employee.Id });
        }

        // …а читаем в совершенно новом — данные взяты из базы, а не из памяти
объекта
        using (var db = _testDb.CreateContext())
        {
            var product = new ProductService(db).GetById(productId);
            Assert.NotNull(product);
            Assert.Equal("Дырокол", product!.Name);
            Assert.Equal(2, new StockService(db).GetBalance(productId));
            Assert.Equal(2, new StockService(db).GetHistory(new HistoryFilter
{ ProductId = productId }).Count);
        }
    }
}
```

Приложение Б

Перечень автоматических тестов

Таблица 10 – Автоматические тесты (полный перечень)

№	Название теста	Функция	Исходные данные	Ожидаемый результат	Результат
1	РасчетОстатка_ПоступленияИВыдачи_РавенРазнице	Ф4	+100, +20, −30, −25	остаток 65	пройден
2	РасчетОстатка_НетОпераций_РавенНулю	Ф4	нет операций	остаток 0	пройден
3	РасчетОстатка_ВыданоВсё_РавенНулю	Ф4	+10, −10	остаток 0	пройден
4	КонтрольМинимума_РазныеОстатки_ВерныйПризнак (4 случая)	Ф6	остаток/минимум: 4/5, 5/5, 6/5, 0/0	true, true, false, true	пройден
5	ВыдатьТовар_ВПределахОстатка_УспешноОстатокУменьшился	Ф3	остаток 10, выдача 4	успех, остаток 6	пройден
6	ВыдатьТовар_БольшеОстатка_ОтклоняетсяОстатокНеИзменился	Ф3	остаток 10, выдача 15	отказ «Недостаточно товара», остаток 10	пройден
7	ВыдатьТовар_РовноВесьОстаток_ДопустимоОстатокНоль	Ф3	остаток 10, выдача 10	успех, остаток 0	пройден
8	ВыдатьТовар_ОстатокПлюсОдин_Отклоняется	Ф3	остаток 10, выдача 11	отказ	пройден
9	ВыдатьТовар_БезСотрудника_Отклоняется	Ф3	получатель не указан	отказ	пройден
10	ВыдатьТовар_КоличествоНеПоложительное_Отклоняется (2 случая)	Ф3	количество 0 и −5	отказ	пройден
11	ВыдатьТовар_ДатаВБудущем_Отклоняется	Ф3	дата – завтра	отказ	пройден
12	ВыдатьТовар_АрхивныйТовар_Отклоняется	Ф3	товар в архиве	отказ	пройден
13	ВыдатьТовар_НеизвестныйТовар_ЗаписьНеНайдена	Ф3	товар Id = 999	отказ, IsNotFound	пройден
14	ВыдатьТовар_НеизвестныйСотрудник_ЗаписьНеНайдена	Ф3	сотрудник Id = 999	отказ, IsNotFound	пройден
15	ВыдатьТовар_ОстатокОпустилсяДоМинимума_УспехСПредупреждением	Ф6	остаток 10, минимум 3, выдача 7	успех и предупреждение	пройден
16	ВыдатьТовар_ОстатокВышеМинимума_БезПредупреждения	Ф6	остаток 10, минимум 3, выдача 6	успех без предупреждения	пройден

№	Название теста	Функция	Исходные данные	Ожидаемый результат	Результат
17	Поступление_КорректныеДанные_ОстатокУвеличился	Ф2	поступление 40	остаток 40	пройден
18	Поступление_БезПоставщикаИДокумента_Отклоняется	Ф2	поставщик из пробелов	отказ	пройден
19	Поступление_ТолькоНомерДокумента_Допустимо	Ф2	только номер документа	успех	пройден
20	Поступление_НаАрхивныйТовар_Отклоняется	Ф2	товар в архиве	отказ	пройден
21	Поступление_КоличествоНоль_Отклоняется	Ф2	количество 0	отказ	пройден
22	Поступление_НеизвестныйТовар_ЗаписьНеНайдена	Ф2	товар Id = 999	IsNotFound	пройден
23	СоздатьТовар_КорректныеДанные_Сохраняется	Ф1	«kt-020», « Ластик »	сохранён как «КТ-020», «Ластик»	пройден
24	СоздатьТовар_ПовторяющийсяАртикул_Отклоняется	Ф1	«kt-001» при существующем «КТ-001»	отказ	пройден
25	СоздатьТовар_ПустоеНаименование_Отклоняется	Ф1	наименование из пробела	отказ	пройден
26	СоздатьТовар_ОтрицательныйМинимум_Отклоняется	Ф1	минимум −1	отказ	пройден
27	СоздатьТовар_МинимумНоль_Допустимо	Ф1	минимум 0	успех	пройден
28	ИзменитьТовар_НеизвестныйId_Отклоняется	Ф1	Id = 999	«Товар не найден»	пройден
29	АрхивироватьТовар_Повторно_Отклоняется	Ф1	два архивирования подряд	первое – успех, второе – отказ	пройден
30	НовыйТовар_БезОпераций_ОстатокНольИТребуетПополнения	Ф4, Ф6	товар без операций, минимум 5	остаток 0, «Требует пополнения»	пройден
31	Поиск_ПоЧастиНаименованияБезУчетаРегистра_НаходитТовары	Ф1	запрос «РУЧКА»	найдено 2 товара	пройден
32	Поиск_ПоАртикулу_НаходитОдинТовар	Ф1	запрос «kt-003»	найден «Карандаш НВ»	пройден
33	Поиск_НетСовпадений_ПустойСписок	Ф1	запрос «степлер»	пустой список	пройден
34	ФильтрНижеМинимума_ПоказываетТолькоТоварыСОстаткомНеВышеМинимума	Ф6	остаток = минимум, остаток > минимума, архивный	только первый товар	пройден
35	Справочник_ПоУмолчанию_АрхивныеСкрыты	Ф1	1 активный, 1 архивный	1 по умолчанию, 2 с флагом	пройден

№	Название теста	Функция	Исходные данные	Ожидаемый результат	Результат
36	История_БезФильтра_ВсеОперацииНовыеСверху	Ф5	4 операции	4 записи, первая – 31.10.2026	пройден
37	История_ФильтрПоТовaруИТипу_ТолькоВыдачиРучки	Ф5	товар + тип «Выдача»	1 запись, 30 шт.	пройден
38	История_ФильтрПоСотруднику_ТолькоЕгоВыдачи	Ф5	сотрудник Петров	1 запись	пройден
39	История_ФильтрПоПериоду_ГраницыВключаются	Ф5	02.10–10.10.2026	2 записи (границы включены)	пройден
40	ИтогиЗаПериод_Октябрь_СуммыПоТоварамВерны	Ф7	октябрь 2026	ручка +100/−30, бумага +40/−15	пройден
41	ИтогиЗаПериод_ПериодБезОпераций_ПустойСписок	Ф7	сентябрь 2026	пустой список	пройден
42	СоздатьСотрудника_ПустоеФИО_Отклоняется	Ф8	пустое ФИО	отказ	пройден
43	ИзменитьСотрудника_НеизвестныйId_Отклоняется	Ф8	Id = 999	«Сотрудник не найден»	пройден
44	Сохранение_ЗаписьЧерезСервис_ЧитаетсяНовымКонтекстом	Ф1–Ф5	запись в одном DbContext, чтение в новом	товар, остаток 2 и 2 операции на месте	пройден

Приложение В

Таблица ручных проверок

Таблица 11 – Ручные проверки функций модуля (полная таблица)

№	Функция	Исходные данные и действия	Ожидаемый результат	Фактический результат	Вывод
1	Ф1 Список	открыть «Товары»	11 активных товаров, 4 выделены	Отображено товаров: 11, выделено «Требует пополнения»: 4	выполнено
2	Ф1 Поиск	ввести «ручка», «Найти»	2 ручки	Найдено: Ручка шариковая синяя; Ручка шариковая чёрная	выполнено
3	Ф1 Добавление	добавить КТ-013 «Скотч прозрачный 19 мм»	сообщение об успехе	Товар «Скотч прозрачный 19 мм» добавлен	выполнено
4	Ф1 Повтор артикула	добавить товар с артикулом «kt-001»	отказ	Товар с артикулом «КТ-001» уже существует	выполнено
5	Ф1 Пустое поле	добавить товар без наименования	отказ	Укажите наименование	выполнено
6	Ф1 Изменение	КТ-013: минимум 4 → 6	сохранено	Изменения сохранены / По этому товару ещё не было операций. Остаток — 0.; мин. остаток: 6	выполнено
7	Ф1 Архив	КТ-013 «В архив»	товар в архиве	Товар отправлен в архив / По этому товару ещё не было операций. Остаток — 0.	выполнено
8	Ф2 Поступление	КТ-001 +50, поставщик и ТН-1100	остаток 15 → 65	Поступление оформлено: 50 ед.; остаток КТ-001 стал: 65 шт.	выполнено
9	Ф2 Без поставщика	КТ-002, количество 5, поставщик и документ пусты	отказ	Для поступления укажите поставщика или номер документа	выполнено
10	Ф3 Сверх остатка	КТ-002 (остаток 38), выдать 100	отказ	Недостаточно товара «Ручка шариковая чёрная»: на складе 38 шт., запрошено 100 шт.	выполнено

№	Функция	Исходные данные и действия	Ожидаемый результат	Фактический результат	Вывод
11	Ф3 Без получателя	КТ-002, выдать 1, сотрудник не выбран	отказ	Для выдачи укажите сотрудника-получателя	выполнено
12	Ф3 Дата в будущем	КТ-002, дата – завтра	отказ	Дата операции не может быть в будущем	выполнено
13	Ф3 Количество 0	в форме и через API	отказ	форма: Браузер не отправил форму (поле «Количество» допускает минимум 1); API: HTTP 400: {"error":"Количество должно быть целым числом от 1 до 100000"}	выполнено
14	Ф3, Ф6 До минимума	КТ-005 (16 уп., минимум 3), выдать 13	успех с предупреждением	Выдача оформлена: 13 ед. / Внимание: остаток «Скрепки 28 мм, 100 шт.» — 3 уп., минимальный — 3. Товар требует пополнения.; остаток КТ-005: 3 уп.	выполнено
15	Ф3 Весь остаток	КТ-006 (4 шт.), выдать 4	успех, остаток 0	Выдача оформлена: 4 ед. / Внимание: остаток «Степлер №24/6» — 0 шт., минимальный — 2. Товар требует пополнения.; остаток КТ-006: 0 шт.	выполнено
16	Ф3 Архивный товар	форма выдачи; API-выдача КТ-011	нет в списке; отказ	КТ-011 в списке формы выдачи: нет; HTTP 400: Нельзя проводить операции с архивным товаром «Корректирующая лента»	выполнено
17	Ф4 Карточка	открыть КТ-001	остаток 15 шт., 4 операции	Остаток в карточке КТ-001: 15 шт.; строк истории: 4	выполнено
18	Ф5 Фильтр истории	товар КТ-001, тип «Выдача»	3 выдачи, −85	Найдено операций: 3. Поступило: +0, выдано: −85.	выполнено
19	Ф5 Период	01.10–15.10.2026	операции периода	Найдено операций: 16. Поступило: +420, выдано: −102.	выполнено

№	Функция	Исходные данные и действия	Ожидаемый результат	Фактический результат	Вывод
20	Ф6 Ниже минимума	открыть «Ниже минимума»	4 товара	Показаны: Ручка шариковая синяя; Бумага офисная А4, 500 л.; Папка-скоросшиватель пластик; Клей-карандаш 21 г	выполнено
21	Ф7 Итоги	октябрь 2026	суммы по товарам	Итого (в единицах учёта) +420 −177	выполнено
22	Ф7 Неверный период	начало позже конца	сообщение об ошибке	Дата начала периода не может быть позже даты окончания	выполнено
23	Ф8 Сотрудники	добавить сотрудника	сообщение об успехе	Сотрудник «Фёдорова Наталья Юрьевна» добавлен	выполнено
24	Неизвестный Id	/Products/ Details/999 и /api/products/999	404 без падения	HTTP 404: Запись не найдена (404); HTTP 404: Товар с Id = 999 не найден	выполнено
25	Сохранение после перезапуска	остановить и снова запустить приложение, найти КТ-013, открыть КТ-001	данные на месте	КТ-013 найден: да (Скотч прозрачный 19 мм); остаток КТ-001 после перезапуска: 65 шт.	выполнено

Приложение Г

Листинг ключевых классов

Файл Services/StockService.cs

```
using Microsoft.EntityFrameworkCore;
using StationeryAccounting.Data;
using StationeryAccounting.Models;
using StationeryAccounting.ViewModels;

namespace StationeryAccounting.Services;

/// <summary>
/// Движение товаров: поступление, выдача, расчёт остатка, история и итоги.
/// Здесь находится главное правило модуля — нельзя выдать больше, чем есть на
складе.
/// </summary>
public class StockService
{
    // Общий замок на всё приложение: две выдачи одного товара, пришедшие
одновременно,
    // выполняются по очереди. Иначе обе могли бы увидеть один и тот же остаток
    // и вместе выдать больше, чем есть.
    private static readonly object IssueLock = new();

    private readonly AppDbContext _db;

    public StockService(AppDbContext db)
    {
        _db = db;
    }

    /// <summary>Текущий остаток товара по формуле Σ поступлений − Σ
выдач.</summary>
    public int GetBalance(int productId)
    {
        var operations = _db.StockOperations
            .AsNoTracking()
            .Where(o => o.ProductId == productId)
            .ToList();
        return StockCalculator.CalculateBalance(operations);
    }

    /// <summary>Оформить поступление товара.</summary>
    public OperationResult Receive(ReceiptForm form)
    {
        var commonError = CheckQuantityAndDate(form.Quantity, form.Date);
        if (commonError != null)
        {
            return OperationResult.Fail(commonError);
        }

        string? supplier = Clean(form.Supplier);
        string? document = Clean(form.DocumentNumber);
        if (supplier == null && document == null)
        {
            return OperationResult.Fail("Для поступления укажите поставщика или
номер документа");
        }

        var product = _db.Products.FirstOrDefault(p => p.Id == form.ProductId);
```

```
        if (product == null)
        {
            return OperationResult.NotFound("Товар не найден");
        }
        if (product.IsArchived)
        {
            return OperationResult.Fail($"Нельзя проводить операции с архивным
товаром «{product.Name}»");
        }

        _db.StockOperations.Add(new StockOperation
        {
            ProductId = product.Id,
            Type = OperationType.Receipt,
            Quantity = form.Quantity,
            Date = form.Date.Date,
            Supplier = supplier,
            DocumentNumber = document,
            Comment = Clean(form.Comment),
            CreatedAt = DateTime.Now
        });
        _db.SaveChanges();
        return OperationResult.Ok();
    }

    /// <summary>
    /// Оформить выдачу товара сотруднику. Отклоняется, если товара не хватает.
    /// Если после выдачи остаток ≤ минимума — выдача проходит, но возвращается
предупреждение.
    /// </summary>
    public OperationResult Issue(IssueForm form)
    {
        var commonError = CheckQuantityAndDate(form.Quantity, form.Date);
        if (commonError != null)
        {
            return OperationResult.Fail(commonError);
        }

        if (form.EmployeeId == null)
        {
            return OperationResult.Fail("Для выдачи укажите сотрудника-
получателя");
        }

        var product = _db.Products.FirstOrDefault(p => p.Id == form.ProductId);
        if (product == null)
        {
            return OperationResult.NotFound("Товар не найден");
        }
        if (product.IsArchived)
        {
            return OperationResult.Fail($"Нельзя проводить операции с архивным
товаром «{product.Name}»");
        }

        var employee = _db.Employees.FirstOrDefault(e => e.Id ==
form.EmployeeId);
        if (employee == null)
        {
            return OperationResult.NotFound("Сотрудник не найден");
        }

        lock (IssueLock)
        {
```

```
            // ГЛАВНОЕ ПРАВИЛО: нельзя выдать больше, чем есть на складе
            int balance = GetBalance(product.Id);
            if (form.Quantity > balance)
            {
                string unit = Units.Short(product.Unit);
                return OperationResult.Fail(
                    $"Недостаточно товара «{product.Name}»: на складе {balance}
{unit}, " +
                    $"запрошено {form.Quantity} {unit}");
            }

            _db.StockOperations.Add(new StockOperation
            {
                ProductId = product.Id,
                Type = OperationType.Issue,
                Quantity = form.Quantity,
                Date = form.Date.Date,
                EmployeeId = employee.Id,
                Comment = Clean(form.Comment),
                CreatedAt = DateTime.Now
            });
            _db.SaveChanges();

            // Контроль минимального количества
            int newBalance = balance - form.Quantity;
            string? warning = null;
            if (StockCalculator.NeedsRestock(newBalance, product.MinQuantity))
            {
                warning = $"Внимание: остаток «{product.Name}» — {newBalance}
{Units.Short(product.Unit)}, " +
                          $"минимальный — {product.MinQuantity}. Товар требует
пополнения.";
            }
            return OperationResult.Ok(warning);
        }
    }

    /// <summary>История движения с фильтрами. Новые операции —
сверху.</summary>
    public List<StockOperation> GetHistory(HistoryFilter filter)
    {
        // IQueryable — запрос ещё не выполнен, условия добавляются по одному
        IQueryable<StockOperation> query = _db.StockOperations
            .AsNoTracking()
            .Include(o => o.Product)
            .Include(o => o.Employee);

        if (filter.ProductId != null)
            query = query.Where(o => o.ProductId == filter.ProductId);
        if (filter.Type != null)
            query = query.Where(o => o.Type == filter.Type);
        if (filter.EmployeeId != null)
            query = query.Where(o => o.EmployeeId == filter.EmployeeId);
        if (filter.From != null)
            query = query.Where(o => o.Date >= filter.From.Value.Date);
        if (filter.To != null)
            query = query.Where(o => o.Date <= filter.To.Value.Date);

        return query
            .OrderByDescending(o => o.Date)
            .ThenByDescending(o => o.Id)
            .ToList();
    }
```

```
    /// <summary>Итоги за период по каждому товару: сколько поступило и сколько
выдано.</summary>
    public List<TotalsRow> GetTotals(DateTime from, DateTime to)
    {
        var operations = _db.StockOperations
            .AsNoTracking()
            .Include(o => o.Product)
            .Where(o => o.Date >= from.Date && o.Date <= to.Date)
            .ToList();

        return operations
            .GroupBy(o => o.ProductId)
            .Select(g => new TotalsRow
            {
                ProductId = g.Key,
                Article = g.First().Product!.Article,
                Name = g.First().Product!.Name,
                Unit = g.First().Product!.Unit,
                Received = g.Where(o => o.Type == OperationType.Receipt).Sum(o
=> o.Quantity),
                Issued = g.Where(o => o.Type == OperationType.Issue).Sum(o =>
o.Quantity)
            })
            .OrderBy(r => r.Article)
            .ToList();
    }

    // Общие проверки для поступления и выдачи
    private static string? CheckQuantityAndDate(int quantity, DateTime date)
    {
        if (quantity <= 0)
            return "Количество должно быть целым числом больше 0";
        if (quantity > 100000)
            return "Количество не может быть больше 100000";
        if (date.Date > DateTime.Today)
            return "Дата операции не может быть в будущем";
        return null;
    }

    // Пустая строка или пробелы → null, иначе обрезаем пробелы по краям
    private static string? Clean(string? value)
    {
        return string.IsNullOrWhiteSpace(value) ? null : value.Trim();
    }
}
```

Файл Services/StockCalculator.cs

```
using StationeryAccounting.Models;

namespace StationeryAccounting.Services;

/// <summary>
/// Расчёты по складу без обращения к базе — «чистые» функции,
/// которые удобно проверять автотестами.
/// </summary>
public static class StockCalculator
{
    /// <summary>
    /// Текущий остаток: Σ поступлений − Σ выдач.
    /// Для товара без операций остаток равен 0.
    /// </summary>
    public static int CalculateBalance(IEnumerable<StockOperation> operations)
    {
        int received = operations
```

```
            .Where(o => o.Type == OperationType.Receipt)
            .Sum(o => o.Quantity);

        int issued = operations
            .Where(o => o.Type == OperationType.Issue)
            .Sum(o => o.Quantity);

        return received - issued;
    }

    /// <summary>
    /// Товар требует пополнения, если остаток меньше или РАВЕН минимальному.
    /// Пример: минимум 5, остаток 5 — уже пора заказывать.
    /// </summary>
    public static bool NeedsRestock(int balance, int minQuantity)
    {
        return balance <= minQuantity;
    }
}
```

Файл Services/OperationResult.cs

```
namespace StationeryAccounting.Services;

/// <summary>
/// Результат действия в сервисе. Вместо исключений на каждую ошибку
пользователя
/// сервис возвращает понятный ответ: успешно или нет, и текст ошибки по-русски.
/// </summary>
public class OperationResult
{
    public bool Success { get; private set; }

    /// <summary>Текст ошибки, если действие отклонено.</summary>
    public string? Error { get; private set; }

    /// <summary>Предупреждение при успешном действии (например, остаток ниже
минимума).</summary>
    public string? Warning { get; private set; }

    /// <summary>True, если ошибка в том, что запись не найдена (для ответа 404
в API).</summary>
    public bool IsNotFound { get; private set; }

    public static OperationResult Ok(string? warning = null)
    {
        return new OperationResult { Success = true, Warning = warning };
    }

    public static OperationResult Fail(string error)
    {
        return new OperationResult { Success = false, Error = error };
    }

    public static OperationResult NotFound(string error)
    {
        return new OperationResult { Success = false, Error = error, IsNotFound
= true };
    }
}
```

Приложение Д

Инструкция запуска

Необходимые средства: .NET SDK 10.0 (проверка – команда dotnet --version, версия должна начинаться с 10), Git (для получения проекта из репозитория), браузер, подключение к интернету при первой сборке (скачиваются пакеты NuGet) и для оформления страниц (Bootstrap).

Получение проекта: распаковать архив проекта в любую папку или выполнить команду git clone https://github.com/Black287234/KP-Vla.git.

Запуск: открыть папку проекта в терминале и выполнить команду dotnet run --project StationeryAccounting. Дождаться строки «Now listening on: http://localhost:5080» и открыть этот адрес в браузере. В Visual Studio 2026 – открыть файл StationeryAccounting.sln и нажать F5. Остановить приложение – клавиши Ctrl+C.

Подготовка базы данных: отдельно устанавливать SQLite не нужно. При первом запуске приложение само применяет миграцию InitialCreate, создаёт файл StationeryAccounting/stationery.db и заполняет его учебными данными. Альтернативный способ – команда dotnet ef database update (требуется инструмент dotnet-ef: dotnet tool install --global dotnet-ef). Чтобы вернуть учебные данные, нужно остановить приложение, удалить файл stationery.db и запустить приложение снова.

Запуск тестов: в корневой папке выполнить команду dotnet test. Ожидаемый результат – «Passed: 48, Failed: 0».

Проверка основного сценария: открыть «Товары» и убедиться, что выводятся остатки и выделены товары, требующие пополнения; оформить поступление на товар КТ-001; оформить выдачу товара КТ-002 в количестве 100 и убедиться, что выдача отклонена с сообщением «Недостаточно товара»; открыть «История» и отфильтровать операции по товару.

Приложение Е

Инструкция пользователя

Назначение модуля. Модуль предназначен для кладовщика или офис-менеджера и позволяет вести справочник канцелярских товаров, оформлять поступления и выдачи, видеть текущий остаток, историю движения и товары, которые нужно пополнить.

Основные экраны. Меню в верхней части страницы содержит разделы: «Товары» – справочник с остатками; «Ниже минимума» – товары, которые пора заказать; «Поступление» и «Выдача» – формы операций; «История» – все операции с фильтрами; «Итоги» – суммы за период; «Сотрудники» – справочник получателей.

Как оформить выдачу. Открыть раздел «Выдача» или нажать кнопку «− Выдача» в карточке товара. Выбрать товар и сотрудника, указать количество и дату, при необходимости – комментарий. Нажать «Оформить выдачу». После успешной выдачи откроется карточка товара с новым остатком.

Как оформить поступление. Открыть раздел «Поступление», выбрать товар, указать количество, дату и поставщика или номер документа. Нажать «Оформить поступление».

Как исправить ошибочную операцию. Операции не редактируются и не удаляются. Если выдача оформлена ошибочно, нужно оформить поступление этого же товара на то же количество с комментарием «Корректировка: …».

Значение сообщений:

– «Недостаточно товара …: на складе N, запрошено M» – на складе меньше товара, чем указано; уменьшите количество или оформите поступление;

– «Внимание: остаток … Товар требует пополнения» – выдача выполнена, но товар пора заказать;

– «Для выдачи укажите сотрудника-получателя» – не выбран сотрудник;

– «Для поступления укажите поставщика или номер документа» – оба поля пусты;

– «Дата операции не может быть в будущем» – выбрана дата позже сегодняшней;

– «Нельзя проводить операции с архивным товаром» – товар в архиве, верните его из архива в карточке товара;

– «Товар с артикулом … уже существует» – введите другой артикул;

– «Запись не найдена (404)» – открыта ссылка на несуществующий товар.